



Observability Guide

Observability Guidance for NVIDIA Enterprise Reference Architectures

ERA-00007-001-V03

2026-February

NVIDIA Enterprise Reference Architecture Notice & Disclaimers

NVIDIA Enterprise Reference Architecture documentation is NVIDIA confidential information and embodies substantial intellectual property and engineering know-how. NVIDIA grants you limited permissions to use NVIDIA Enterprise Reference Architectures to create NVIDIA platform-enabled data center clusters with NVIDIA technologies. You may not use the NVIDIA Enterprise Reference Architectures to develop competing products or technologies or assisting a third party in such activities. Except as expressly stated in this Notice & Disclaimer statement, no other license or right is granted to you by implication, estoppel or otherwise. NVIDIA objects to and rejects any additional or different terms you may propose.

NVIDIA Enterprise Reference Architectures are subject to change without notice. NVIDIA technologies may require enabled hardware, software, or service activation. Your costs and results may vary. This document is not a commitment to develop, release, or deliver any technology, code, or functionality. NVIDIA reserves the right to make corrections, modifications, enhancements, improvements, and any other changes to this document, at any time without notice. It is your sole responsibility to evaluate and determine the applicability of any information contained in this document and ensure that the product is suitable and fit for the application planned by you. Information published by NVIDIA regarding third-party products or services does not constitute a license from NVIDIA to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party or a license from NVIDIA.

NVIDIA products or services are subject to the applicable NVIDIA product or sales terms and conditions that are provided in conjunction with the relevant NVIDIA product or service. NVIDIA's provision of these resources does not expand or otherwise alter NVIDIA's applicable warranties or warranty disclaimers for the NVIDIA products or services.

You will not use the NVIDIA Enterprise Reference Architecture documentation or NVIDIA's confidential information to: (i) identify or support an assertion or potential assertion of any intellectual property rights (including patent, copyright, or trade secret); or (ii) prepare, file, amend, or prosecute any patent applications. You agree to grant NVIDIA a limited, worldwide, nonexclusive, perpetual, irrevocable, non-sublicensable, nontransferable, royalty-free, fully-paid license to any patent claim that you may file after accessing NVIDIA Enterprise Reference Architecture that covers the subject matter disclosed herein.

Warranty Disclaimer. NVIDIA Enterprise Reference Architecture materials disclosed are provided "AS IS". To the maximum extent permitted by applicable law, NVIDIA disclaims all warranties and representations of any kind, whether express, implied, or statutory, relating to or arising under this agreement, including, without limitation, the warranties of title, noninfringement, merchantability, fitness for a particular purpose, usage of trade, and course of dealing.

Limitation of Liability. To the maximum extent permitted by applicable law, in no event will NVIDIA be liable for any (i) indirect, punitive, special, incidental, or consequential damages, or (ii) damages for the (a) cost of procuring substitute goods or (b) loss of profits, revenues, use, data, or goodwill arising out of or related to this document, whether based on breach of contract, tort (including negligence), strict liability, or otherwise, and even if NVIDIA has been advised of the possibility of such damages and even if a your remedies fail their essential purpose. Additionally, to the maximum extent permitted by applicable law, NVIDIA's total cumulative aggregate liability for any and all liabilities, obligations, or claims arising out of or related to this document will not exceed one-hundred U.S. dollars (US\$100).

NVIDIA, NVIDIA HGX, NVIDIA DGX SuperPOD, NVIDIA DGX Cloud, NVIDIA ConnectX, NVIDIA BlueField, NVIDIA Spectrum, NVIDIA NVLink, NVIDIA Base Command, NVIDIA CUDA, NVIDIA Magnum IO, NVIDIA NetQ, and the NVIDIA logo are trademarks and/or registered trademarks of NVIDIA Corporation in the U.S. and other countries. Other company and product names may be trademarks of the respective companies with which they are associated.

© 20256 NVIDIA Corporation & Affiliates. All rights reserved.

Shashank Sabhlok

Abstract

This Observability Guide for NVIDIA Enterprise Reference Architectures (Enterprise RAs) offers a standardized and production-ready reference as well as step-by-step guidance for implementing observability in enterprise AI or HPC environments.

Built on top of NVIDIA's AI infrastructure and Kubernetes-native platforms, this version of the guide specifically focuses on advanced custom dashboard and alerting solutions for AI factories, providing administrators and enterprise customers with actionable insights into GPU, CPU, Kubernetes, and applications. This includes guidance on setting up data pipelines and custom dashboards using tools such as NVIDIA Data Center GPU Manager (DCGM), NIM Operator, Prometheus Node Exporter, and other telemetry sources. Dashboards and alerts will ensure improved cluster health visibility as it experiences AI workloads and accelerated troubleshooting.

NVIDIA global system partners, channel partners, and enterprise customers are invited to apply the guidance described in this document to their software stack of choice, and to substitute components (e.g. from NVIDIA ecosystem software partners offering Kubernetes distributions and/or observability and monitoring tools) where needed while retaining the benefits of the observability approach.

Future versions of this guide will extend observability capabilities to include logging and distributed tracing, enabling a complete end-to-end observability stack.

Contents

Overview of the Observability Guide for NVIDIA Enterprise Reference Architectures (Enterprise RAs)	1
Technical Prerequisites for Observability	3
<i>NVIDIA Enterprise RAs – Prerequisites</i>	<i>3</i>
Verifying Kube Prometheus and Grafana Setup	3
Configuring Persistent Storage for Grafana	5
Steps to configure a PVC for Grafana	6
Network Monitoring with NVIDIA NetQ	8
Configuration of Data Sources	14
<i>Base Command Manager – Prometheus</i>	<i>14</i>
<i>DCGM Exporter</i>	<i>19</i>
<i>NIM Metrics – Kube Prometheus</i>	<i>20</i>
<i>NetQ Prometheus</i>	<i>24</i>
Custom Dashboards	25
<i>GPU Dashboard</i>	<i>25</i>
<i>NIMs Dashboard</i>	<i>27</i>
<i>Kubernetes Global Dashboard</i>	<i>29</i>
<i>CPU & Node Storage Dashboard</i>	<i>31</i>
<i>Network Dashboard</i>	<i>33</i>
Alerting.....	36
<i>Configuring Grafana for Alerting.....</i>	<i>36</i>
Folder Creation	36
Contact Point Creation	37
Setting up SMTP Server for configuration	38
<i>Automated Alert Uploads</i>	<i>40</i>
Script Overview	41
<i>Alert Rule JSONs.....</i>	<i>44</i>
Summary	49
Out of Scope	50
Appendix A.....	51

Overview of the Observability Guide for NVIDIA Enterprise Reference Architectures (Enterprise RAs)

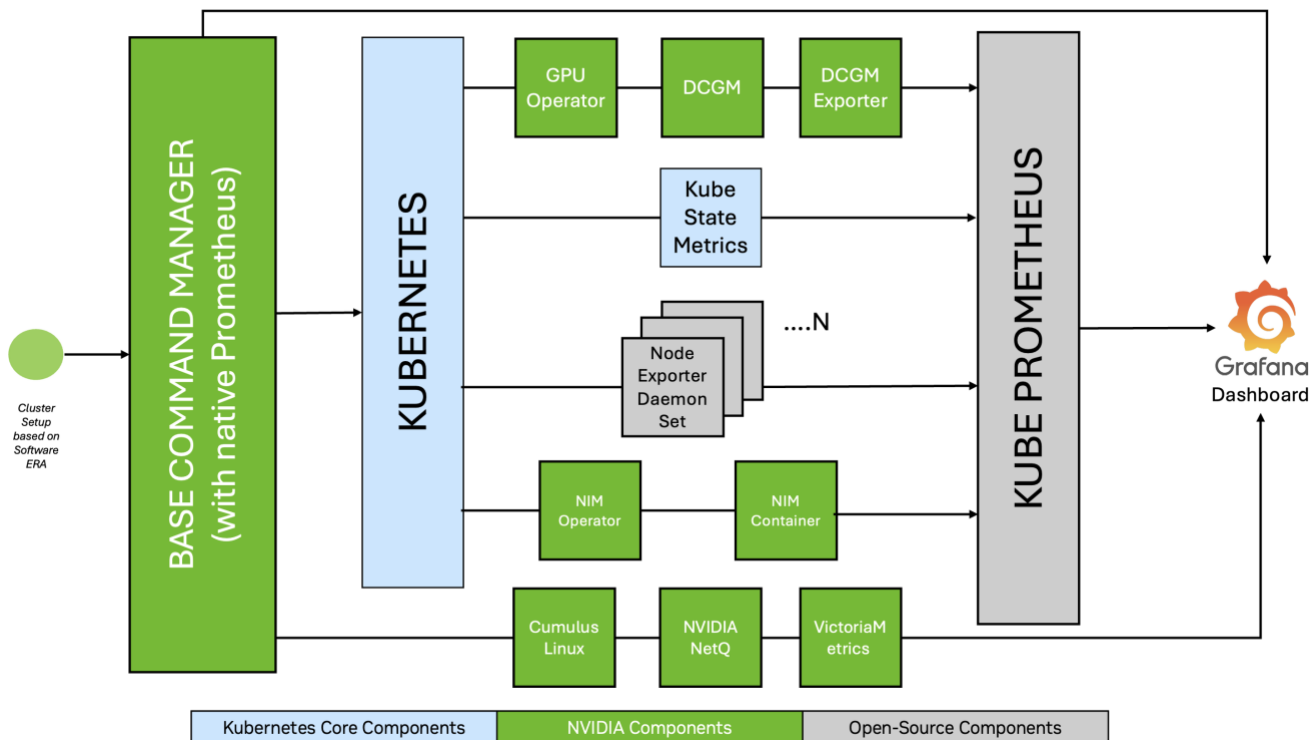
The Observability Guide for NVIDIA Enterprise RAs is designed to provide customers with a methodology to monitor the health, performance and resource utilization of NVIDIA's AI infrastructure starting from Day 1.

As AI workloads scale and grow in complexity, observability becomes a critical requirement to ensure system reliability, and resource efficiency. This Observability Guide complements the existing hardware-focused NVIDIA Enterprise RAs, and is intended to accelerate time-to-value for customers and partners by reducing the operational setup associated with observability tools from ecosystem software partners. Along with this guide, there are Grafana JSON dashboard and alerting templates that can be accessed through [GitHub](#).

This Observability Guide is intended to be used as a design reference and practical step-by-step guide for developers building an observability stack on Enterprise RAs. While this guide offers a reference, it is not exhaustively prescriptive. Organizations can substitute components for functional equivalents and maintain the benefits of enabling observability across the stack.

For example, various Kubernetes distribution options (e.g., Red Hat OpenShift or Canonical Kubernetes) are compatible with the components described¹ and recommendations offered in this guide. This guidance is also supplemented by an ecosystem of validated observability software partners – described in the NVIDIA AI factory validated and reference designs for enterprise and government. NVIDIA global system partners and channel partners can apply the observability approach described below to their customers' enterprise stack of choice.

¹ With the exception of NVIDIA Base Command Manager.



Technical Prerequisites for Observability

NVIDIA Enterprise RAs – Prerequisites

This Observability Guide for Enterprise RAs is designed to layer seamlessly on top of NVIDIA's enterprise AI infrastructure, providing visibility into AI workloads running across compute, network and software layers. To get started, it is recommended that customers follow Enterprise RA hardware infrastructure guidance for NVIDIA accelerated computing with NVIDIA Spectrum-X networking to power the next generation of data centers architecture for AI and HPC workloads.

This guide builds upon the foundation provided by the guidance in the Software Reference Stack and Automation guide for NVIDIA Enterprise RAs (as indicated in the diagram above) which offers a modular and optional software stack for deploying and managing AI workloads at scale. Core components of this Software Reference Stack – such as GPU Operator, Network Operator and Prometheus ensure that the infrastructure and workloads are ready for deep observability. Moving forward in this document, it is assumed that the software stack prescribed has been deployed and tested across nodes.



Note: For detailed overview of these software components please refer to the Software Reference Stack and Automation for NVIDIA Enterprise RAs | NVOnline: 1130533

Verifying Kube Prometheus and Grafana Setup

For the purposes of visualizations and alerting, we will be leveraging Prometheus as a time series database and Grafana for dashboarding and data visualization. The setup wizard for NVIDIA Base Command Manager (BCM) facilitates the installation of the Kube Prometheus stack including Prometheus operator, Node Exporter, Kube State Metrics and Grafana. Please refer to Appendix A for details on which port to keep open for these tools. You can verify the setup with the following Kubernetes commands:

Example 1. Commands for verifying deployments

```
root@pdxera-bcm01:~# kubectl get ns
```

NAME	STATUS	AGE
cm	Active	150d
cm-permissions	Active	134d
cmkpm-system	Active	134d
default	Active	150d
gpu-operator	Active	150d
ingress-nginx	Active	150d
kube-node-lease	Active	150d
kube-public	Active	150d
kube-system	Active	150d
kubernetes-dashboard	Active	150d
metallb-system	Active	150d
network-operator	Active	150d
nfs-provisioner	Active	121d
nim-operator	Active	126d
prometheus	Active	150d
web	Active	89d

```
root@pdxera-bcm01:~$ kubectl get deployments -n prometheus
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
kube-prometheus-stack-grafana	1/1	1	1	141d
kube-prometheus-stack-kube-state-metrics	1/1	1	1	141d
kube-prometheus-stack-operator	1/1	1	1	141d
prometheus-adapter	1/1	1	1	141d

```
root@pdxera-bcm01:~$ kubectl get serviceMonitor -n prometheus
```

NAME	AGE
kube-prometheus-stack-alertmanager	141d
kube-prometheus-stack-apiserver	141d
kube-prometheus-stack-coredns	141d
kube-prometheus-stack-grafana	141d
kube-prometheus-stack-kube-controller-manager	141d
kube-prometheus-stack-kube-etcd	141d
kube-prometheus-stack-kube-proxy	141d
kube-prometheus-stack-kube-scheduler	141d
kube-prometheus-stack-kube-state-metrics	141d

kube-prometheus-stack-kubelet	141d
kube-prometheus-stack-operator	141d
kube-prometheus-stack-prometheus	141d
kube-prometheus-stack-prometheus-node-exporter	141d



Note: For guidance on external access setup of Kube Prometheus and Grafana please refer to NVIDIA's Software Reference Stack and Automation for Enterprise RAs | NV Online: 1130533

Configuring Persistent Storage for Grafana

By default, Grafana stores its dashboards, data sources, user settings, plugins in the `/var/lib/grafana` directory inside the pod. If this directory is not backed with persistent storage, all configurations and user data will be lost whenever the pod is rescheduled, the node reboots, or the deployment is updated. For production environments, it's important to ensure data persistence, as metrics and dashboards are typically captured over weeks and months.

To ensure data durability during pod restarts and deployment rollouts, it is recommended that a Persistent Volume Claim (PVC) is attached to the Grafana instance. The PVC can leverage your existing enterprise storage infrastructure (in our case a NFS system) to provide persistent storage.

Steps to configure a PVC for Grafana

To start, ensure a Storage Class is configured with suitable persistent backend storage. In the example below our Storage Class is called `nfs-client` and a separate monitoring namespace has been created for Grafana and Kube Prometheus. Once declared, create a PVC with sufficient storage to retain the Grafana data over time.

Example 2. Manifest to create PVC for Grafana

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: grafana-data
  namespace: monitoring
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: nfs-client
```

This PVC can be applied to your cluster by saving it to a file (e.g. `pvc.yaml`) and running the Kubernetes apply command. Once created, verify that it is correctly bound to the Persistent Volume (PV):

Example 3. Commands to apply and verify the PVC

```
root@pdxera-bcm01:~$ kubectl apply -f pvc.yaml
root@pdxera-bcm01:~$ kubectl get pvc -n monitoring
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS	MODES	STORAGECLASS
VOLUMEATTRIBUTESCLASS		AGE				
elasticsearch	Bound	pvc-3042c813	5Gi	RWO		local-path
<unset>	70d					
grafana-data	Bound	pvc-1b30e4e1	5Gi	RWX		nfs-client
<unset>		39d				

You should see the PVC status as **Bound**.

The next step is to modify the existing Grafana Deployment to use the PVC. Specifically, add a `volumeMounts` entry in the Grafana container specification to mount the PVC to `/var/lib/Grafana`, and add a corresponding `volumes` entry to the pod specification pointing to the PVC. Below is an example from our Grafana deployment:

Example 4. Extract from a Grafana deployment manifest with changes to incorporate a PV

```
...

volumeMounts:
- mountPath: /tmp/dashboards/dcgm.json
  name: nv-dashboard-volume
  subPath: dcgm.json
- mountPath: /etc/grafana/grafana.ini
  name: config
  subPath: grafana.ini
- mountPath: /var/lib/grafana
  name: grafana-storage

...

volumes:
- configMap:
  defaultMode: 420
  name: nvidia-dcgm
  name: nv-dashboard-volume
- configMap:
  defaultMode: 420
  name: kube-prometheus-stack-grafana
  name: config
- name: grafana-storage
  persistentVolumeClaim:
    claimName: grafana-data
- emptyDir: {}
  name: sc-dashboard-volume
- configMap:
  defaultMode: 420
```

```

    name: kube-prometheus-stack-grafana-config-dashboards
  name: sc-dashboard-provider
- emptyDir: {}
  name: sc-datasources-volume

```

Finally, you can verify if the PVC is correctly mounted by exec'ing into the Grafana pod:

Example 5. Commands to execute commands inside the Grafana Pod

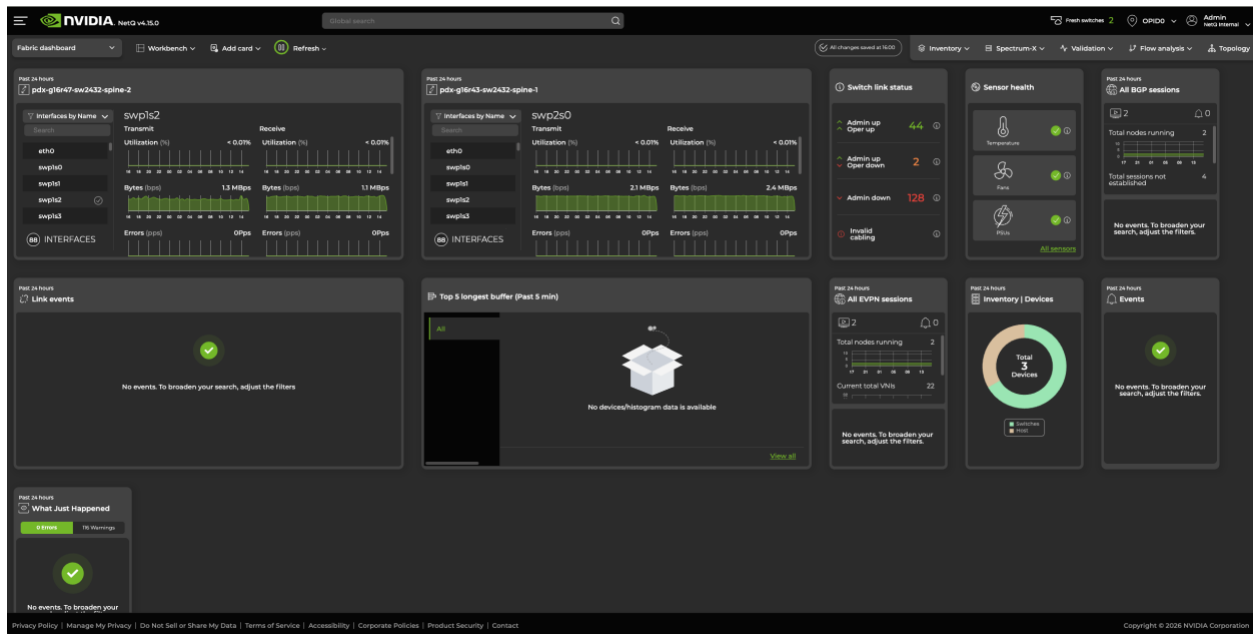
```

root@pdxera-bcm01:~$ kubectl exec -it grafana-pod -n monitoring -- sh

/usr/share/grafana $ df -h | grep /var/lib/grafana
                    5.7T          1.7T          3.7T   32% /var/lib/grafana

```

Network Monitoring with NVIDIA NetQ



NVIDIA NetQ is NVIDIA's network operations and telemetry platform for Cumulus Linux-based switches and NVIDIA network adapters. It collects detailed statistics about links, routing, NVSwitch, and platform health, and makes them available to external tools such as Prometheus and Grafana. To install NetQ on your control node, and configure NetQ CLI/Agents on your switches follow the instructions [here](#). This guide assumes that a NetQ server is installed and reachable by the switches, which in turn have NetQ CLI and NetQ agents running on them.



Note: For more information on configuring Spectrum switches and setting up VRFs please refer to the Networking Configuration & Logical Architectural Guide for NVIDIA Enterprise RAs | NVOnline: 1133364 is for the 2-4-3-200 node architecture

The following commands configure the NetQ agent on the switch to send telemetry, statistics, What Just Happened (WJH) events and ASIC monitoring data to the NetQ server using the management VRF so the data can be displayed on the NetQ UI as shown above. Note, the agent needs to be restarted so all these settings take effect.

Example 6. NetQ agent configuration to export switch telemetry to the NetQ UI

```
# Configure NetQ agent to use server 10.XXX.XXX.XXX over the mgmt VRF
netq config add agent server 10.XXX.XXX.XXX vrf mgmt

# Enable NetQ agent statistics collection and export
netq config add agent stats

# Configure a CPU usage limit for the NetQ agent (value typically follows)
netq config add agent cpu-limit

# Enable What Just Happened (WJH) event collection in the NetQ agent
netq config add agent wjh

# Enable ASIC monitoring in the NetQ agent
netq config add agent asic-monitor

# Restart the NetQ agent to apply configuration changes
netq config restart agent
```

Each switch in your setup must export OpenTelemetry (OTel) metrics to the NetQ server. This is configured through the following NVUE commands on Cumulus Linux. For this guide, the Spectrum switches are running Cumulus Linux 5.12

Example 7. Commands to enable and verify telemetry on the networking switch

```
# Turn global telemetry on
nv set system telemetry enable on

# Write histogram stats snapshots to a memory backed file
nv set system telemetry snapshot-file name /var/run/cumulus/histogram_stats
```

```
# Keep upto 120 histogram snapshot files
nv set system telemetry snapshot-file count 120

# Take a histogram snapshot every 10 seconds
nv set system telemetry snapshot-interval 10

# Enable OTLP export so telemetry is actually sent
nv set system telemetry export otlp state enabled

# Send telemetry using the mgmt VRF
nv set system telemetry export vrf mgmt

# Set OTLP gRPC collector destination to 10.XXX.XXX.XXX on port 30008
nv set system telemetry export otlp grpc destination 10.XXX.XXX.XXX port 30008

# Use insecure (non-TLS) gRPC for OTLP export
nv set system telemetry export otlp grpc insecure enabled

# Enable exporting of buffer usage statistics via telemetry
nv set system telemetry buffer-stats export state enabled

# Enable exporting of control-plane statistics via telemetry
nv set system telemetry control-plane-stats export state enabled

# Enable export of histogram-style telemetry statistics
nv set system telemetry histogram export state enabled

# Collect/export egress buffer stats for traffic class 0,3,6,12 to gauge priority
nv set system telemetry interface-stats egress-buffer traffic-class 0
nv set system telemetry interface-stats egress-buffer traffic-class 3
nv set system telemetry interface-stats egress-buffer traffic-class 6
nv set system telemetry interface-stats egress-buffer traffic-class 12

# Enable exporting of per-interface telemetry statistics
nv set system telemetry interface-stats export state enabled
```

```

# Enable low-level physical interface telemetry class
nv set system telemetry interface-stats class phy state enabled

# Collect/export ingress buffer stats for priority groups 0,1, 4 on interfaces
nv set system telemetry interface-stats ingress-buffer priority-group 0
nv set system telemetry interface-stats ingress-buffer priority-group 1
nv set system telemetry interface-stats ingress-buffer priority-group 4

# Set telemetry interface stats sample interval to 30 seconds
nv set system telemetry interface-stats sample-interval 30

# Enable exporting of platform statistics (CPU, memory, sensors, etc.)
nv set system telemetry platform-stats export state enabled

# Enable exporting of BGP routing telemetry statistics
nv set system telemetry router bgp export state enabled

# Enable exporting of general routing telemetry statistics
nv set system telemetry router export state enabled

# Enable exporting of Routing Information Base (RIB) telemetry statistics
nv set system telemetry router rib export state enabled

# Enable detailed environment sensor platform telemetry (temps, fans, PSUs)
nv set system telemetry platform-stats class environment-sensor state enabled

# Define snapshot group to capture all per-port packet statistics
nv set system telemetry snapshot port-group packet-all-pg interface all

# Include all packet counters in the snapshot group
nv set system telemetry snapshot port-group packet-all-pg stats-type packet-all

# Capture packet snapshots every 20 seconds
nv set system telemetry snapshot port-group packet-all-pg timer-interval 20

# Write packet snapshot data to /var/lib/cumulus/all_packet_stats
nv set system telemetry snapshot port-group packet-all-pg snapshot-file name
/var/lib/cumulus/all_packet_stats

```

```

# Keep up to 120 packet snapshot files
nv set system telemetry snapshot port-group packet-all-pg snapshot-file count 120

# Apply the configuration
nv config apply -y

# Show system configuration
nv show system telemetry

# Show OTLP gRPC export destination for system telemetry
nv show system telemetry export otlp grpc destination

# Show system telemetry health status
nv show system telemetry health

```

Replace 10.XXX.XXX.XXX with the IP of your NetQ server. Port 30008 is the default OTLP port used by NetQ for metrics ingestion so please ensure that it is updated.

As shown in the example below, a healthy output should show nv-telemetry-service, routing-telemetry-service, and platform-stats-service in active state along with an “Export Destination Status” table with the NetQ server IP and port, Connectivity: OK, and non-zero export counters.

Example 8. Health output that verifies system telemetry is sent to the NetQ Server IP.

```

nv show system telemetry health
operational
-----
service-status
  nv-telemetry-service      active
  platform-stats-service    active
  histogram-export-service  active
  sdk-stats-service         active
  routing-telemetry-service active
internal-metrics
  process
    cpu-seconds             24449
    memory-rss-kilobytes    111648768
    runtime-heap-alloc-bytes 35428336
    runtime-total-alloc-bytes 4548898418280
    runtime-total-sys-memory-bytes 57826320
    uptime-seconds          531056

```



```

[receivers]                                otlp/global
[receivers]                                prometheus/global
processors
  [memory-limiter]                         memory_limiter/1
  [batch]                                   batch/1
[exporters]                                otlp/global/1

```

Export Destination Status

```

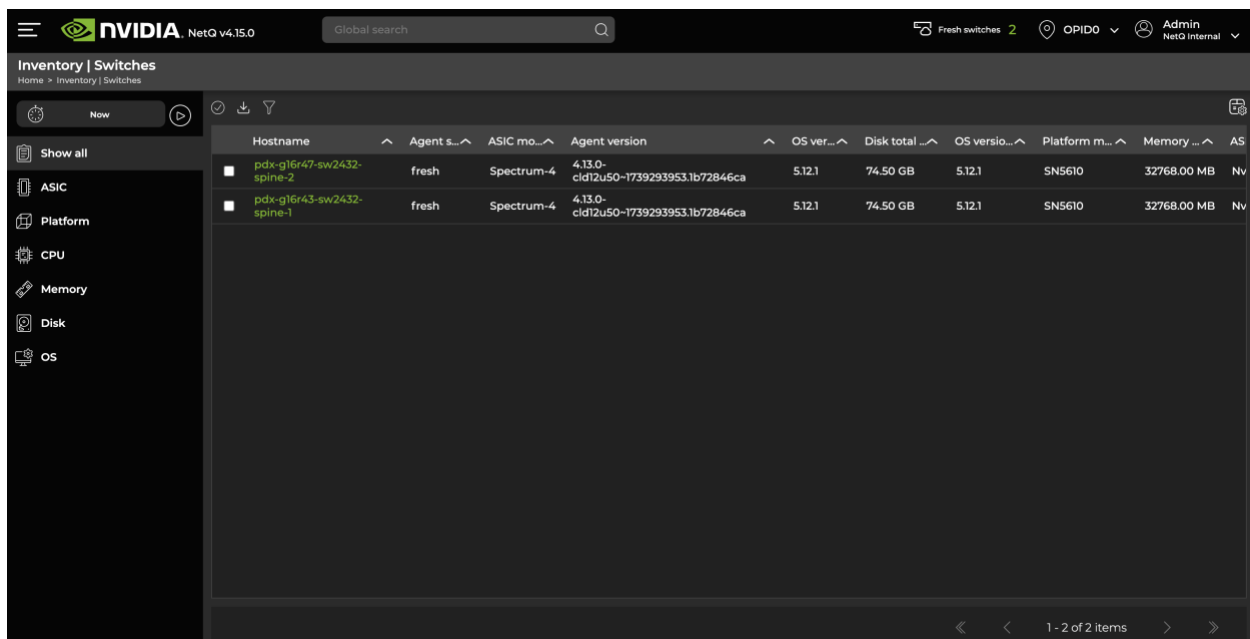
=====

```

Destination	Connectivity	Export Counter	Drop Counter
-----	-----	-----	-----
10.XXX.XXX.XXX:30008	Pass	3099406634	0

NetQ can accept OTLP traffic in secure (TLS) or insecure mode. In many NetQ deployments it is simple from a setup standpoint to start with an insecure deployment and enable TLS later. On the NetQ server, use the netq CLI to alter the OLTP security mode: `sudo netq set otlp security-mode insecure`

Once OTLP export and security are aligned, confirm that the NetQ agent and telemetry pipeline are healthy. On the switches run `netq config status agent verbose`. A healthy output shows NetQ subagents (netlink process, gRPC stream, ASIC monitor) marked as alive. Furthermore, on the NetQ UI navigate to Inventory -> Switches and verify that all the spines appear with status fresh – this confirms NetQ is receiving basic telemetry from the devices even before Grafana is connected.



Configuration of Data Sources

Base Command Manager – Prometheus

NVIDIA Base Command Manager (BCM) offers out-of-the-box telemetry for monitoring and managing compute clusters. It leverages a native Prometheus instance that integrates smoothly with Grafana, enabling centralized visualization of relevant metrics alongside broader infrastructure monitoring. This instance of Prometheus is installed in the BCM head node.

To establish connectivity, Grafana requires authentication to access metrics from BCM. The recommended approach is certificate-based authentication, which ensures secure and controlled access to the cluster management API. The first step involves defining a management profile as a root user in BCM so that you have access to the right cluster management API calls that are necessary for Grafana to retrieve monitoring data, maintaining security best practices.

Example 9. Commands to define a management profile as a root user in BCM

```
root@pdxera-bcm01:~# cmsh
[pdxera-bcm01]% profile
[pdxera-bcm01->profile]% add grafana
[pdxera-bcm01->profile*[grafana*]]% set services cmmon
[pdxera-bcm01->profile*[grafana*]]% set tokens private_monitoring_token plot_token
get_labeled_entity_token
[pdxera-bcm01->profile*[grafana*]]% commit
[pdxera-bcm01->profile[grafana]]% quit
```

Once the profile is defined, run the show command to confirm it's been created as desired.

Example 10. Command to confirm if a profile has been created

```
[pdxera-bcm01->profile] % use grafana
[pdxera-bcm01->profile[grafana]] % show
Parameter          Value
-----
Name                grafana
Revision
Nonuser            no
Services           CMMon
Tokens             PRIVATE_MONITORING_TOKEN,PLOT_TOKEN,GET_LABELED_ENTITY_TOKEN
```

Subsequently, we will issue a certificate with this profile, which will be leveraged by Grafana to authenticate into the BCM API.

Example 11. Commands to create certificates in BCM as a root user

```
root@ pdxera-bcm01:~# cmsh
[pdxera-bcm01]% cert
[pdxera-bcm01->cert]% createcertificate
Name:                createcertificate - Create a new certificate
Usage:               createcertificate <key-length> <common-name> <organization>
<organizational-unit> <locality> <state> <country> <profile> <sys-login> <days>
<key-file> <cert-file>
Arguments:
    key-file    Path to key file that will be generated
    cert-file   Path to pem file that will be generated

[pdxera-bcm01->cert]% createcertificate 2048 "Grafana" "Bright Computing"
"Engineering" "Amsterdam" "NH" "NL" grafana nobody 3650 /root/grafana.key
/root/grafana.pem

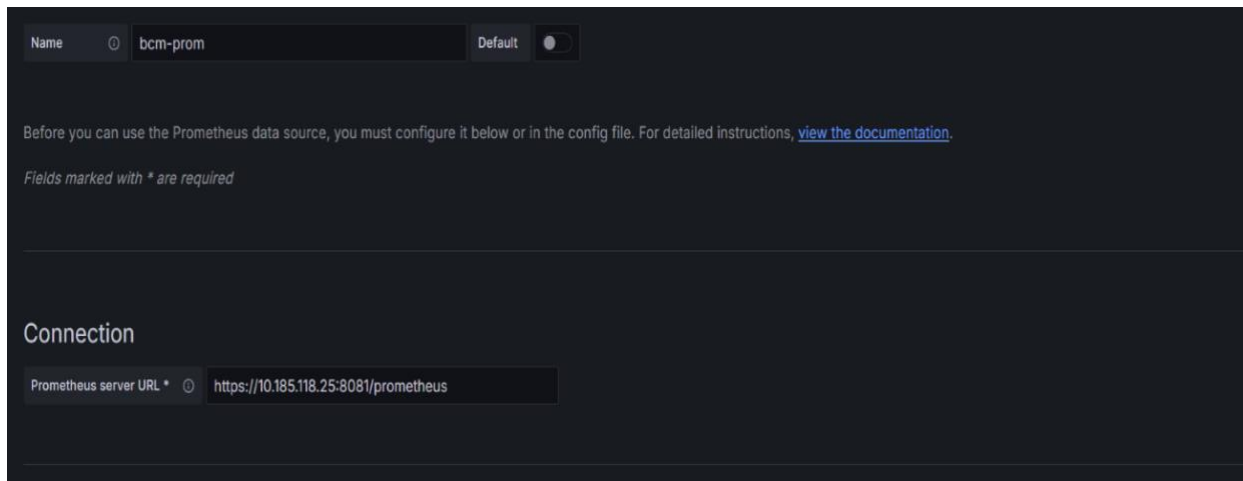
Tue Apr 12 21:58:52 2025 [notice] mdv-test: New certificate request with ID: 2
Certificate key written to file: /root/grafana.key Certificate pem written to file:
/root/grafana.pem
```

We now have a key in /root/grafana.key and a certificate in /root/grafana.pem which will be needed in the next step as we configure Grafana. We will also need the CA certificate, which can be found in /cm/local/apps/cmd/etc/cacert.pem. Note that you will find two certificates in the cacert.pem. You will need to paste both certificates as the CA cert value in setting up BCM as a data source for Grafana.

Example 12. Example of the cacert.pem file

```
-----BEGIN CERTIFICATE-----
<alphanumeric certificate characters> <----- Certificate
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
<alphanumeric certificate characters> <----- Root Certificate
-----END CERTIFICATE-----
```

With authentication in place, navigate to Grafana and select Data Sources under Connections and add a new source of type Prometheus. Assign a meaningful name such as bcm-prom to easily identify the BCM data stream. Provide the Prometheus server URL, ensuring that **HTTPS** is used for the connection to maintain encrypted communication.



The screenshot shows the Grafana web interface for configuring a new Prometheus data source. At the top, the 'Name' field is set to 'bcm-prom' and the 'Default' toggle is turned off. Below this, a message states: 'Before you can use the Prometheus data source, you must configure it below or in the config file. For detailed instructions, [view the documentation](#).' A note below that says 'Fields marked with * are required'. The 'Connection' section is expanded, showing the 'Prometheus server URL *' field with the value 'https://10.185.118.25:8081/prometheus'.

Under the authentication settings, paste the chain of CA certificates, client certificates and private key into the respective fields to complete the setup.

TLS Client Auth

☒

With CA Cert

☒

Skip TLS Verify

☒

Forward OAuth Identity

☐

TLS Auth Details

CA Cert

```

xpJ6r5AnhC9GQuvupkNf4VFY3kPCeXMehCnGVU1v2XahG0kMAJHFpI7dWOubpC
XfUccrAIAANSySGztMJsJkPO+myPhnyT8vHc/R3DtbfmLJEYlihVTog+xVGCGb++f
U26vmuzSy5oR+Z5xmg9xo3YavGypWLzdrJNB3ow88ekB+IWRBgc4AV53dZ1almap
xAnrrtnoPWys5jpSaIUU5erPHDhSbkv5SRvrk9kxHHq534nsaOhEY866f78NrLJ
Pg3wlqQVY6Mq+gmsfms/8DSKeMLF0vb2MMvy/FfyRiVfU1kbzQ==
-----END CERTIFICATE-----

```

Client Cert

```

qcCZZbLX1p25rhcywAULRq+KohMSi8YXXIh0Dep8Sn97b1Vp+pCGPUrsl9gYIEpl
5UdrjEeYjrsnaTK5GpvAGW52VY4+UIHQQGsWt4qWVrSPZ00HvE8rJaM/UeVSRDc0g
xNGuGxnRKJFFn8ba8J4EtXrc6At/hwYZLSgDgNX7BcaS1cvXmwlbGqhRumBnHBL9
gjYVRH7H3IJASeVeeMyjLXhUQwsPh8pZMtovyJnQklwMCI2JjXqYBKBcbISXL3zL
J8ovu6qIMfxflJwl+lpqvIFHithq4TCF/LFB6/HLRNNCc8NUrtqc5Q==
-----END CERTIFICATE-----

```

Client Key

```

zTQxF0P/E7CeMmgL82f56srEy5hGqRwvzfPo8ka727q6u5clSJ2YLCd/QZ2hoAEb
4rgcf2Qghl70pBI36o6bCcV9wv3bcjuAftJbWhERAoGAYT+EVr+8tKThnd/iWEox
XDZqE7Brvhzat5CWlb/fes+0Z4iqc0jDeiXDv07EGE6/fUubfL1a+HzaGpzwSZaNB
LGy1zheB4/mPfxKZaBDqEkf7y1SR8cr4qS9yGVZsGv91NysOT44G+axf95HRTdEh
1ba6Be2fEyY6pgMbQQJT574=
-----END PRIVATE KEY-----

```

Custom HTTP Headers

+ Add header

Other

Custom query parameters

Example: max_source_resolution=5m&timeour

HTTP method

GET

Exemplars

+ Add

Delete

Save & test

It is important to adjust the HTTP method to GET - failure to make this change may result in connectivity errors with Grafana. Once done, clicking on the Save & Test button should result in a “Data Source is working” message, confirming that Grafana can now pull telemetry from BCM.

Misc

Disable metrics lookup ⓘ ☐

Custom query parameters ⓘ

Example: max_source_resolution=5m&timeout=10

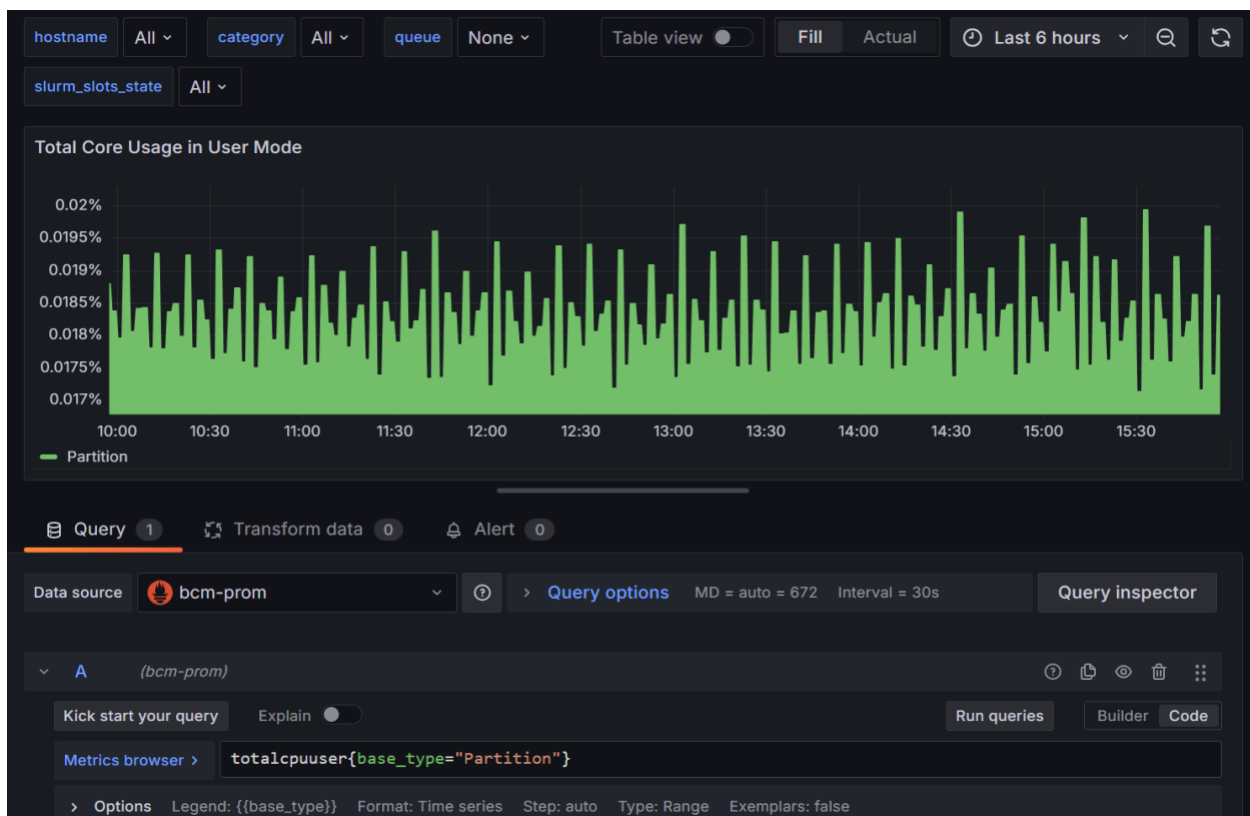
✓ Data source is working

Save & Test

Delete

Back

To validate the metric stream, create a new Grafana visualization using the bcm-prom data source. Navigate to the Dashboards page using the menu pane on the left of the screen. Select **New** and then choose **New dashboard** followed by **Add visualization**. You will be promoted to select a data source where bcm-prom should be an option. After selecting it, you can enter a PromQL query under Query A to validate the data. The example query below shows a Grafana visualization tracking total core CPU usage in user mode.



DCGM Exporter

The NVIDIA GPU Operator simplifies the deployment and monitoring of GPUs in Kubernetes environments by deploying DCGM Exporter alongside NVIDIA Drivers and Kubernetes device plugins. The DCGM Exporter collects real-time GPU telemetry data such as memory usage, temperature and power consumption, and is exposed through a Service Monitor. When Prometheus Operator is deployed alongside the GPU Operator, the exporter is automatically discovered allowing for GPU metric scraping without any manual configuration.



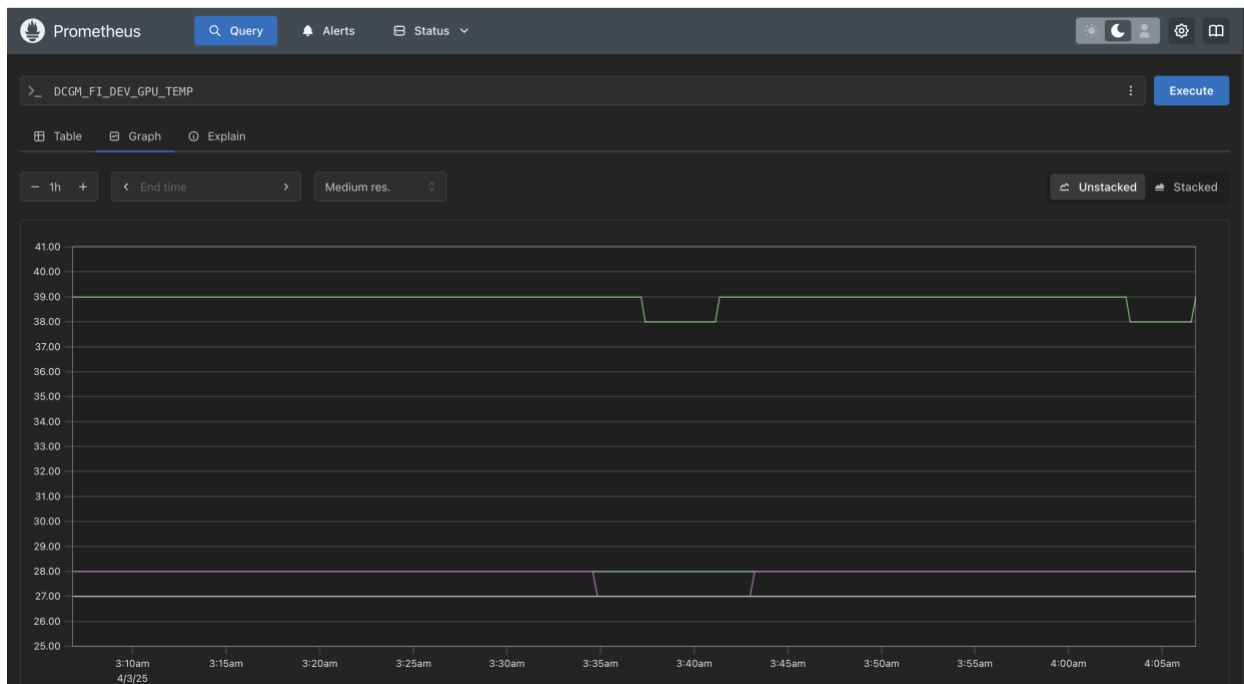
Note: For instructions on how to deploy the GPU Operator or the Prometheus Operator, please refer to the NVIDIA's Software Reference Stack and Automation for Enterprise RAs | NV Online: 1130533

As part of the GPU Operator deployment, DCGM Exporter is also provisioned. Confirm this by verifying that all pods resembling `dcmg-exporter-xxx` are running.

Example 13. Command to verify pods in the GPU Operator namespace

```
kubect1 get pods -n gpu-operator
```

Since metrics should be scraped automatically, verify that you can see DCGM exporter metrics in your instance of Kube Prometheus, by navigating to the Prometheus UI and trying a sample query such as `DCGM_FI_DEV_GPU_TEMP`:



NIM Metrics – Kube Prometheus

The NIM Operator (latest version 3.0.2 as of February 2026) simplifies the deployment and lifecycle management of NVIDIA NIM microservices at the Kubernetes cluster level. It enables streamlined execution of AI inference pipelines, including Retrieval Augmented Generation (RAG) and Multi-LLM workflows, with built in support for model caching to reduce latency.

The section outlines the steps to configure the NIM Operator for Observability, ensuring the NIM metrics can be collected and visualized through Kube Prometheus and Grafana.

Before proceeding further, ensure that you have configured your Kubernetes secret with your NVIDIA GPU Cloud (NGC) API key to authenticate to gain access to the NVIDIA NGC catalog, enabling you to download model, containers and other resources.



Note: : For detailed step-by-step guide to setup NGC API Key, refer to <https://docs.nvidia.com/nim/large-language-models/latest/getting-started.html#export-the-api-key>

Once setup, proceed to configure the NIM cache and NIM service custom resources in Kubernetes. NIM cache is a Kubernetes custom resource that pre-downloads and stores AI model profiles on persistent storage, enabling faster startup for NIM microservices. The moment you create a NIM cache resource, the NIM Operator starts a pod that lists the available model profiles. If you specify one or more model profile IDs to cache, the Operator starts a job that caches the model profiles that you specified. A NIM service is used to launch and serve the cached models. It can be configured to export metrics by setting the `spec.metrics.enabled` flag to true.

Example 14. Manifest for NIM Service

```
apiVersion: apps.nvidia.com/v1alpha1
kind: NIMService
metadata:
  name: meta-llama3-1-8b-instruct
spec:
  image:
    repository: nvcr.io/nim/meta/llama-3.1-8b-instruct
    tag: "1.3"
    pullPolicy: Always
    pullSecrets:
      - ngc-secret
  authSecret: ngc-api-secret
  storage:
```



```

nimCache:
  name: meta-llama3-8b-instruct
  profile: ''
replicas: 1
resources:
  limits:
    nvidia.com/gpu: 1
expose:
  service:
    type: ClusterIP
    port: 8000
metrics:
  enabled: true

```

Apply the manifest:

Example 15. Kubernetes command to create the NIM Service

```
$ kubectl apply -n nim-service -f service.yaml
```

Verify that the Service Monitor has been configured correctly by running the following command:

Example 16. Kubernetes command to verify the Service Monitor

```
kubectl get servicemonitor -n nim-service meta-llama3-1-8b-instruct -o yaml
```

This should return the YAML below. Check the following:

- The port matches the LLM service's exposed metrics port
- Endpoint path is set to /v1/metrics
- Labels match the NIM service labels to facilitate Prometheus association

Example 17. Manifest for the ServiceMonitor that facilitates the scraping

```

apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  annotations:
    nvidia.com/last-applied-hash:
84238486e4b5589a0018bfbea19590c1ecf46999816953bf2b47d5de9b7aea9b
    openshift.io/scc: nonroot
  creationTimestamp: "2025-02-27T05:38:22Z"
  generation: 1

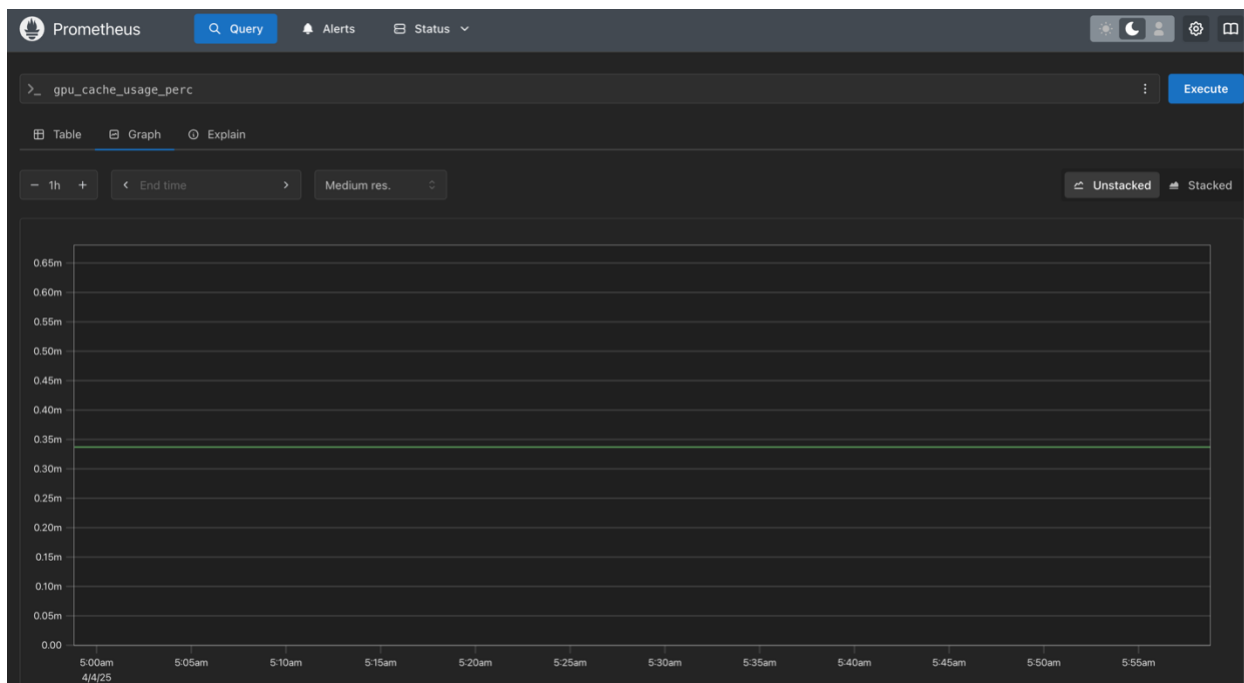
```

```

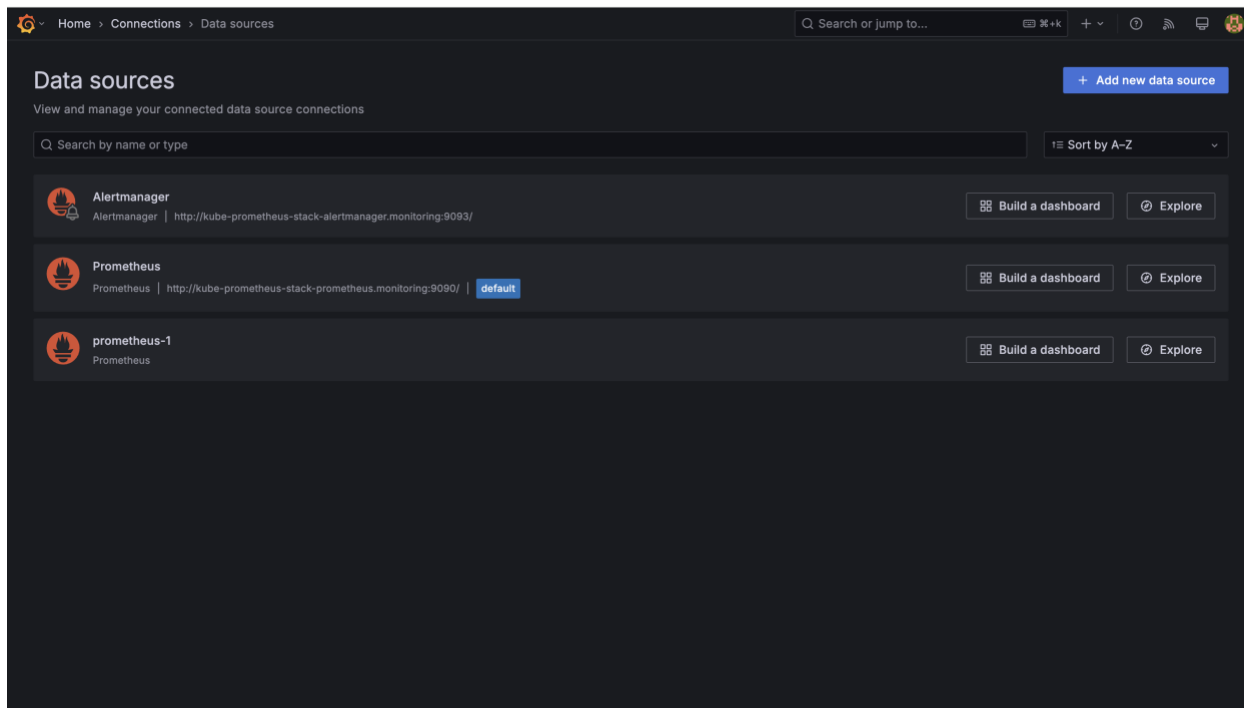
labels:
  app.kubernetes.io/instance: meta-llama3-1-8b-instruct
  app.kubernetes.io/managed-by: k8s-nim-operator
  app.kubernetes.io/name: meta-llama3-1-8b-instruct
  app.kubernetes.io/operator-version: ""
  app.kubernetes.io/part-of: nim-service
name: meta-llama3-1-8b-instruct
namespace: nim-service
ownerReferences:
- apiVersion: apps.nvidia.com/v1alpha1
  blockOwnerDeletion: true
  controller: true
  kind: NIMService
  name: meta-llama3-1-8b-instruct
  uid: b97ea2a5-1021-4851-b244-4f5552bd9493
resourceVersion: "68785693"
uid: fbc999e4-0600-493b-9714-088660d10f32
spec:
  endpoints:
  - path: /v1/metrics
    port: service-port
  namespaceSelector:
    matchNames:
    - nim-service
  selector:
    matchLabels:
      app.kubernetes.io/instance: meta-llama3-1-8b-instruct
      app.kubernetes.io/managed-by: k8s-nim-operator
      app.kubernetes.io/name: meta-llama3-1-8b-instruct
      app.kubernetes.io/operator-version: ""
      app.kubernetes.io/part-of: nim-service

```

Once configured, Kube Prometheus will automatically scrape NIM metrics and expose them for visualization. Below is a Prometheus graph showing the LLM GPU KV Cache usage for a NIM.



Once the data from NIMs and DCGM is present in Kube Prometheus, you should be able to select the Prometheus data source in Grafana, and see the metrics flow through. The link between the Kube Prometheus data source and Grafana should have been established automatically at installation time. Similar to the steps followed above, click on the Kube Prometheus data source, scroll to the bottom and click on Test Connection to verify that the Prometheus API can be queried. A green box should show up saying that the test query was successful.



NetQ Prometheus

Spectrum Switches export OTLP metrics into a time-series database (TSDB) that runs inside the NetQ server. From Grafana's perspective, this TSDB behaves like a Prometheus-compatible endpoint that accepts PromQL-style queries (for example, `nvrouting_*`, `nvswitch_*`).

Similar to how we did for BCM's Prometheus above, we need to add another data source for NetQ Prometheus which can be done through Connections -> Data sources. You may want to give the data source a different name such as `netq-tsdb` or `netq-prom` to differentiate it from `kube-prometheus` or `bcm-prom`.

The Prometheus server URL will be <https://10.XXX.XXX.XXX/api/netq/vm/> (NetQ server from above) where the `/api/netq/vm/` path exposes the NetQ TSDB in a Prometheus-compatible way. To authenticate, configure access using an API token obtained via the NetQ CLI. To obtain the token SSH into the server that is hosting NetQ and run the `netq show vm-token expiry 180` command.

On the same page, navigate to the Authentication section. In the HTTP headers section, select Add Header. In the Header field, enter Authorization and keep the default HTTP method as GET, which is what the NetQ TSDB expects. In the Value field, enter the token that you generated earlier. Ensure 'Skip TLS certificate validation' is checked under TLS Settings.

Click Save & test at the bottom of the page. If the connection is successful, Grafana will report that the data source is working. After this step, the `netq-tsdb` data source becomes available in Grafana's query editor. You can then query network metrics using PromQL-style expressions, for example `node_os_info` to list switches and OS details.



Note: The NetQ UI and Grafana dashboards draw their data from different sources. The NetQ UI is populated from information received via the NetQ agents, while the Grafana dashboards are populated from OTLP telemetry streamed from the switches. As a result, the NetQ UI can show a device as healthy even if Grafana is missing metrics for that device because the OTLP export or TSDB path is misconfigured.

Custom Dashboards

GPU Dashboard

The [GPU Dashboard](#) sources its data from two sources: BCM and NVIDIA DCGM Exporter. BCM provides over-arching health metrics related to GPUs while DCGM metrics provide real-time visibility into the GPU utilization, thermal health and memory usage across a GPU enabled Kubernetes or Slurm cluster. Customers can leverage the GPU Metrics Dashboard to monitor cluster health, identify performance bottlenecks, and troubleshoot issues in production environments. The table below describes each key metric displayed in the dashboard:

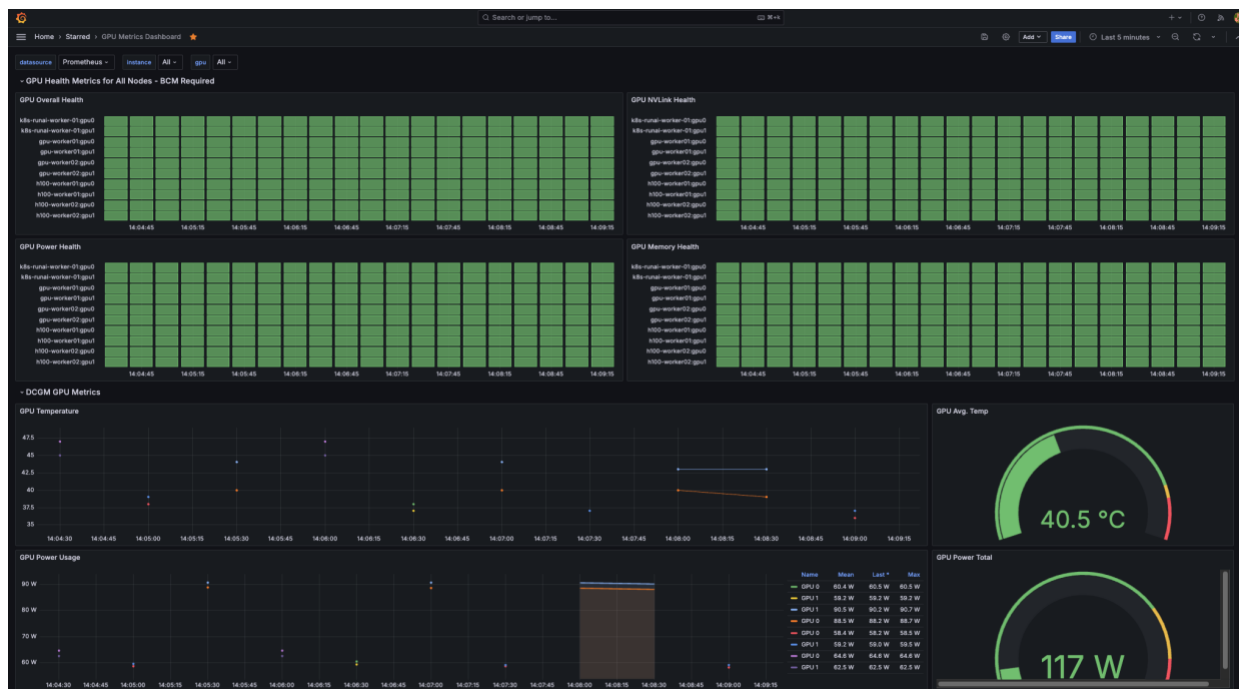
Table 1. Overview of visualizations in the GPU Dashboard

Metric Name	Description
GPU Health Metrics for All Nodes - BCM Required	
GPU Overall Health	Aggregates GPU health parameters to display a quick visual status: green for healthy, red for issues, enabling efficient cluster-wide GPU monitoring and troubleshooting
GPU NVLink Health	Monitors NVLink health for each GPU, showing green for functional links and red for issues
GPU Power Health	Monitors GPU power health, showing green for optimal power levels for the chip and red for anomalies
GPU Memory Health	Monitors GPU memory health where green indicates normal operation and red for issues
DCGM GPU Metrics	
GPU Temperature	Monitors thermal health of the device – temperature of each GPU in Celsius
GPU Average Temperature	Gauge visualization showing average GPU temperature across all GPUs over the selected time window

GPU Power Usage	Time-series plot showing power consumption of each GPU in Watts. Useful for energy efficiency monitoring
GPU Power Total	Gauge showing the total power usage across all GPUs in the cluster at a given time
GPU SM Clocks	Time-series plot of the Streaming Multiprocessor (SM) clock speeds in Hertz (Hz) indicates GPU core frequency
GPU Utilization	Percentage utilization of the GPU cores to measure how busy the GPU is while processing workloads
GPU Framebuffer Memory Used	Tracks the amount of GPU framebuffer memory (VRAM) is currently in use. This is important for memory-bound workload monitoring
Tensor Core Utilization	Percentage of time the GPU Tensor Cores are actively used. This is useful for monitoring AI/ML workloads leveraging tensor cores



Note: Since BCM provides overall metrics, the filters at the top of this Dashboard only apply to the DCGM metrics and not to the BCM metrics



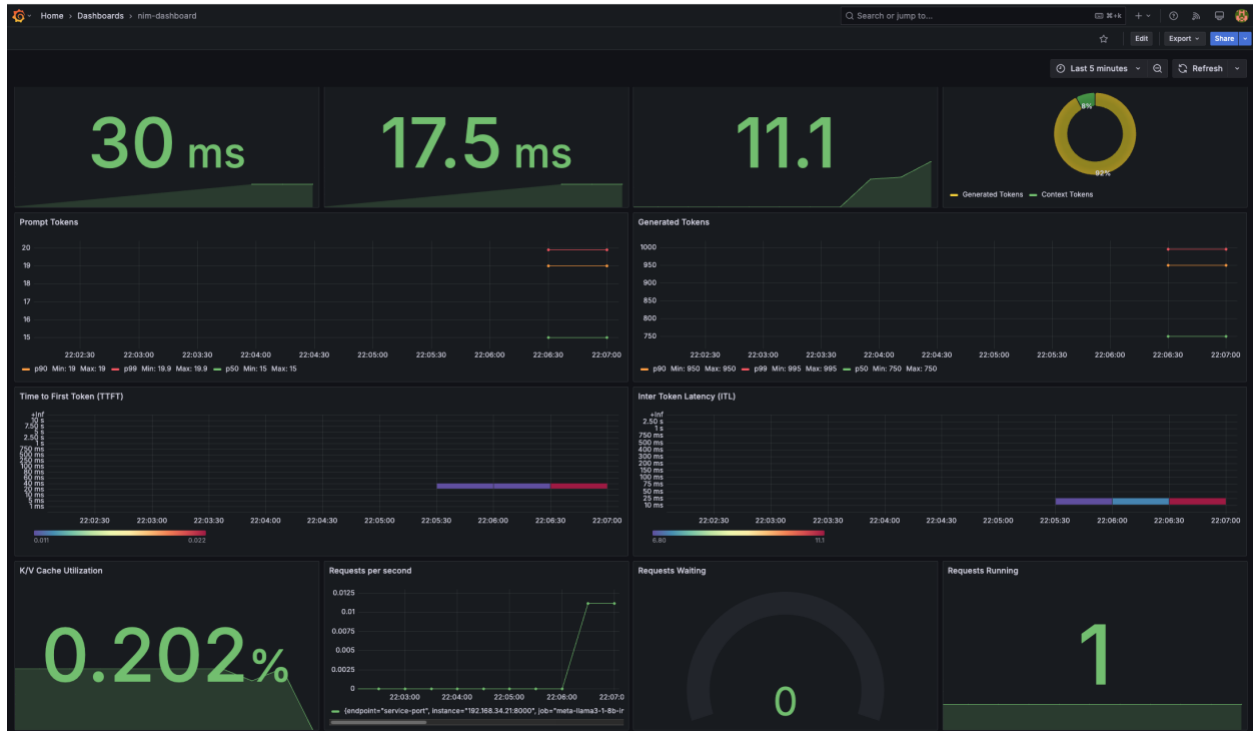
NIMs Dashboard

The [NIMs Dashboard](#) offer enterprise customers visibility into the operational behavior of their LLM inference services, allowing them to tune performance and respond to anomalies swiftly. Metrics such as latency and KV Cache Utilization.

Table 2. Overview of visualizations in the NIMs Dashboard

Metric Name	Description
Time to First Token (TTFT) p50	Median latency between the request submission and the first token being generated – this indicates how quickly the model starts responding
Inter-Token Latency (ITL) p50	Median time delay between consecutive tokens being generated during inference. This helps assess token generation speed
Throughput	Measured in tokens/second, this metric highlights the number of tokens generated per second. Higher throughput indicates efficient model serving
Context versus Generated Tokens	Pie chart that shows the ratio of prompt (context) tokens versus generated tokens in inference

	requests. This helps customers understand the load characteristics
Prompt Tokens	Time-series chart of the number of prompt token received by the model instance per request
Generated Tokens	Time-series chart of the number of tokens generated by the model instance per request
Time to First Token (TTFT) Heatmap	Heatmap showing the spread of TTFT latencies across requests over time. This helps in spotting latency spikes in production
Inter-Token Latency (ITL) Heatmap	Heatmap showing the spread of ITL latencies across requests over time. Helps identify token generation delays
KV Cache Utilization	Percentage of GPU's Key-Value (KV) Cache usage. KV Cache is an optimization where the LLM 'remembers' previous computations in GPU memory. This way for a request prompt or token, the model can generate tokens faster. Higher utilization could indicate memory bottlenecks
Requests per second	Rate of incoming inference requests per second which helps in identifying traffic patterns
Requests Waiting	Number of requests waiting to be processed. Higher values may indicate overloads or bottlenecks
Requests Running	Number of inference requests currently being processed – this helps understand active request concurrency



Kubernetes Global Dashboard

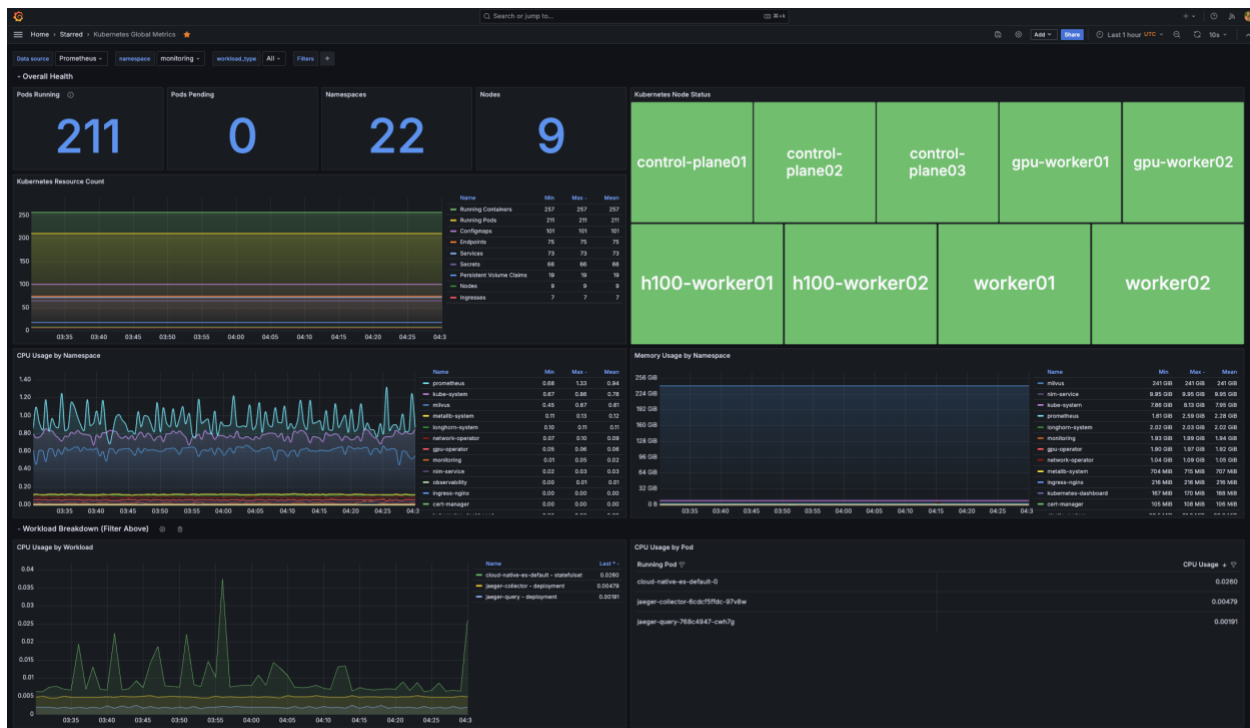
The [Kubernetes Global Dashboard](#) provides a comprehensive view of operational health and performance of the Kubernetes cluster. It consolidates key metrics across cluster resources, workloads, and networking to give operators clear visibility into current state. At a glance, you can monitor the health of core components, including pods, namespaces and nodes. Workload level insights show both CPU and memory consumption across deployments, daemonsets and statefulsets, helping pinpoint high resource consuming services or potential scaling concerns.

The filters in this dashboard do not apply to the “Overall Health” tab and only to the “Workload Breakdown”

Table 3. Overview of visualizations in the Kubernetes Global Dashboard

Metric Name	Description
Overall Health	
Pods Running	Number of currently active pods across all namespaces
Pods Pending	Number of pods in a pending state, awaiting resource allocation or scheduling decisions. High counts may indicate resource bottlenecks

Namespaces	Counts distinct Kubernetes namespaces
Nodes	Number of worker nodes in the Kubernetes cluster
Kubernetes Node Status	Real-time health status of each node. Green signifies nodes are operational and healthy. Red signifies they are not
Kubernetes Resource Count	Snapshot of Kubernetes objects -running pods, config maps, PVCs, nodes and ingresses. Helps tracks the overall cluster composition
CPU Usage by Namespace	Helps identify which teams, users or applications are consuming the most compute resources, allowing for deeper analysis in the Workload Breakdown section
Memory Usage by Namespace	Helps identify which teams, users or applications are consuming the most memory, allowing for deeper analysis in the Workload Breakdown section
Workload Breakdown	
CPU Usage by Workload	Tracks CPU Utilization by each workload (deployment, daemonset, statefulset)
CPU Usage by Pod	Granular view of CPU consumption at the pod level – identifies which specific pods are consuming the most CPU
Memory Usage by Workload	Memory consumption by each workload
Memory Usage by Pod	Memory usage for individual pods, useful for spotting memory-intensive pods that may need reconfiguration



CPU & Node Storage Dashboard

The [CPU & Storage Dashboard](#) provides system-level visibility into CPU performance, memory utilization and storage health of bare-metal nodes. It is designed to give enterprise operators an understanding of CPU and storage related bottlenecks. BCM is leveraged here to provide the overall node health status and hence warrants the BCM Node filter at the top. The CPU metrics and Storage metrics sections rely on the Instance dropdown at the top to filter the graphs.

Table 4. Overview of visualizations in the CPU & Storage Dashboard

Metric Name	Description
Node Health – BCM Required	
Node Status	Based on BCM's device status flag, it gives you the general health of the node
CPU metrics	
Average CPU Utilization	Displays the average percentage of CPU capacity being used across all monitored nodes
Average CPU Temperature	Shows the real-time average temperature of CPU cores across nodes

Average CPU Idle Time	Percentage of time the CPU was idle – higher values indicate less workload on the CPU
Average CPU System Time	Percentage of CPU time spent on kernel level operations
Average CPU User Time	Percentage of CPU time spent on user processes (applications)
Memory Usage	Real-time memory consumption across monitored nodes – helps in identifying memory bottlenecks
Node Storage metrics	
Disk Read Time Latency	Measures the time taken to complete disk read operations for each disk device on the cluster
Disk Write Time Latency	Measures the time taken to complete disk write operations for each disk device on the cluster
Disk I/O Time	Shows the total time spent on I/O operations for each disk device. High I/O time may indicate disk contention
Disk Utilization	Displays the percentage of disk space used across storage devices – helps monitor capacity



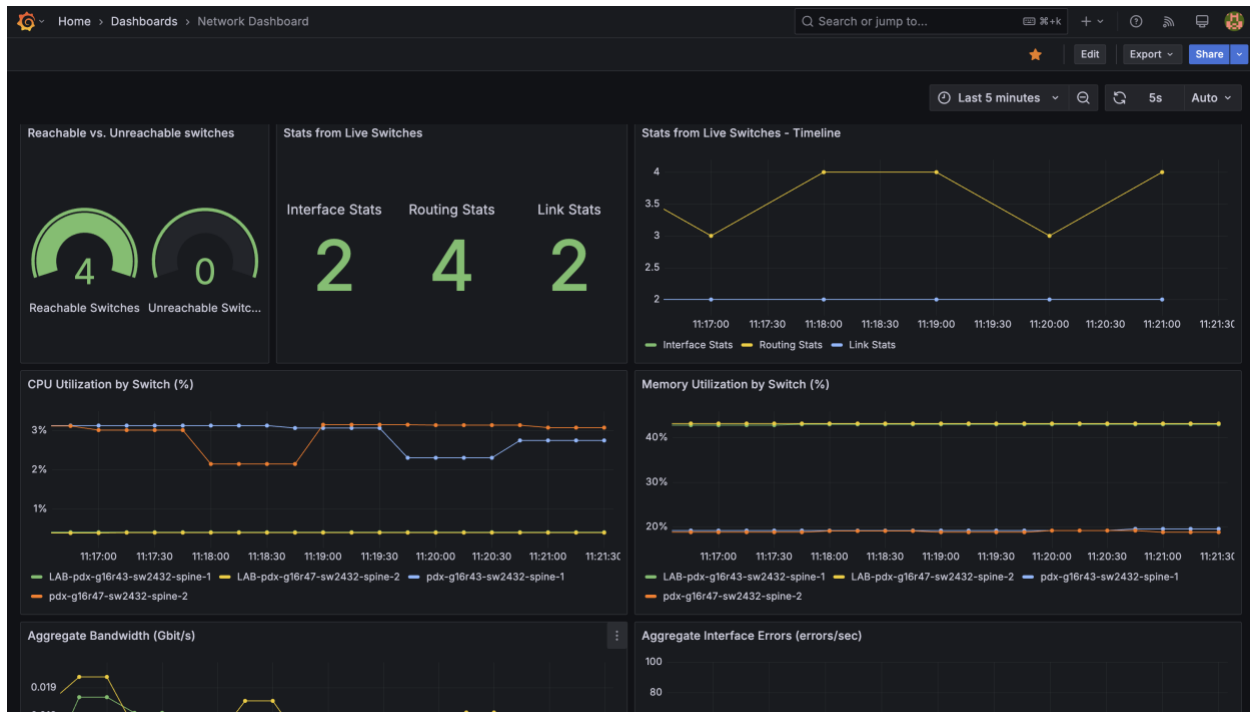
Network Dashboard

The [Network Dashboard](#) provides switch level visibility into link health, traffic patterns, and routing status across the fabric. It is designed to help operators quickly determine whether all spines are exporting telemetry correctly, whether links are up and stable, and whether BGP routing tables are populated as expected. The Inventory section shows which switches are exporting telemetry, their peak port bandwidth, and Cumulus Linux versions. Link Summary panels track link status, link down events, and symbol errors, while Routing panels report BGP peer state and routes per VRF to confirm control plane health.

Metric Name	Description
Switch & Stats Health	
Reachable & Unreachable Switches	Shows how many switches are currently reachable and how many are down so you can see overall fabric availability at a glance.
Stats from Live Switches	The Live Switches panels provide a quick check that telemetry is flowing from every device: Interface Stats shows how many switches export interface metrics, Routing Stats shows how many export BGP and RIB metrics, and Link Stats shows how many export link health metrics

Stats from Live Switches - Timeline	Plots over time how many switches have active Interface, Routing, and Link Stats so you can see gaps in telemetry
Switch & Interface Performance	
CPU Utilization by Switch (%)	Tracks per-switch CPU usage over time to reveal overload or abnormal spikes
Memory Utilization by Switch (%)	Tracks per-switch memory usage over time to catch leaks before it impacts forwarding
Aggregate Bandwidth (Gbit/s)	Shows total transmit bandwidth across all monitored ports on each switch to highlight traffic spikes and sustained load
Aggregate Interface Errors (errors/sec)	Sums interface error rates per switch so you can quickly see if any box is seeing physical or link-layer problems
Interfaces Up per Switch	Shows, per switch, the number of interfaces that are currently up and participating in the network
Link Down Events	Charts the number of link down occurrences over time to reveal flapping links or maintenance windows
Routing Information	
BGP (IPv4 + IPv6) Routes Received per Switch	Shows, per switch, the total number of IPv4, and IPv6 learned from BGP peers that are up
EVPN Prefixes Received	Shows, per switch, how many EVPN control-plane routes (MAC/IP/VNI information) are currently learned from peers
BGP Peer States per Switch	Summarizes, for each switch, how many BGP peers are in Established, Active, or Idle state so you can spot broken sessions
Number of Routes BGP per VRF	Shows how many BGP routes are installed per VRF on each switch to validate routing table population

BGP Peer Status	Summarizes each switch's BGP peers, their current state, and VRF to confirm control-plane health
Switch Software	Lists each switch and its Cumulus Linux version to verify software consistency across the fabric



In a converged network topology (like the one used in creating the dashboard above), traffic between two switches can be validated using `iperf3` to confirm consistent bandwidth and data flow across the fabric. For example, if two servers have IPs `10.183.117.154` and `10.183.117.159`, start the receiver with `iperf3 -s` on `.159`, then run `iperf3 -c 10.184.181.159 -t 600` from `.154` to generate a continuous ten-minute traffic stream between the two endpoints.

During the test, the source console displays transfer rates and bitrates per second, while the Max Port Bandwidth (Gbit/s) panel in Grafana will show corresponding activity spikes, confirming that traffic is being transmitted successfully through and across the switches.

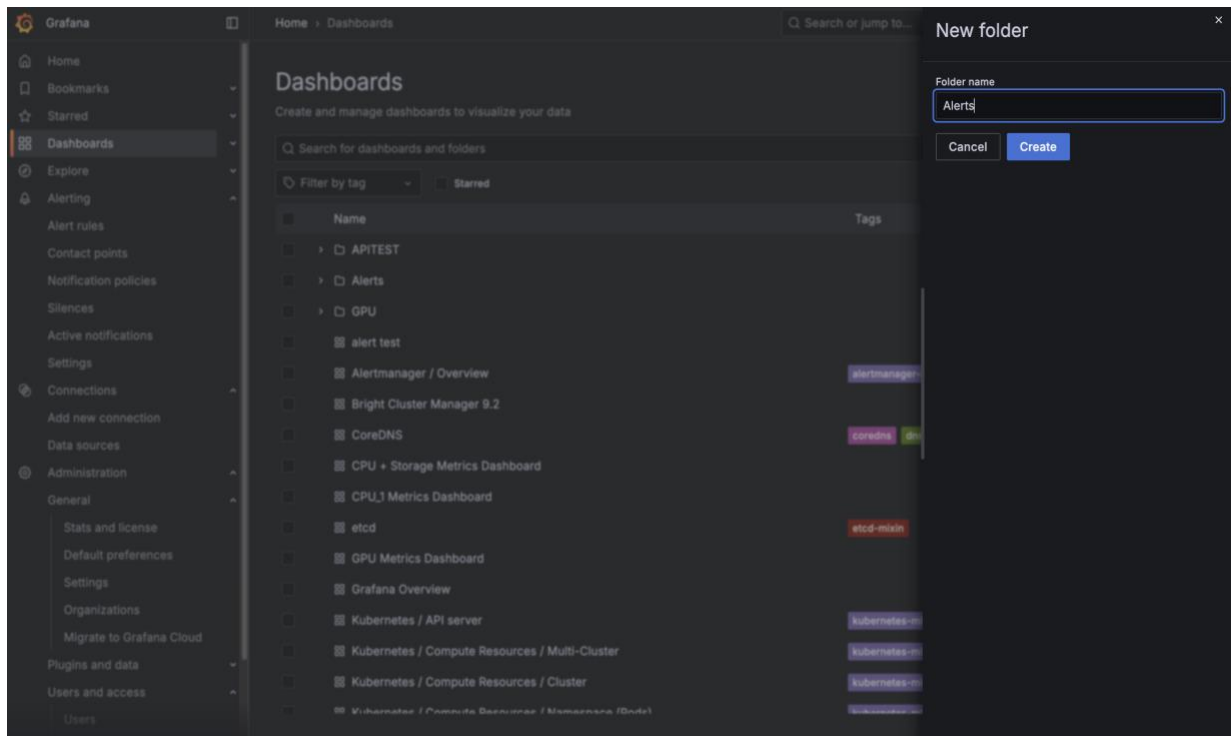
Alerting

Alerting is a critical component of data center operations, providing real-time notifications when teams can respond instantly and prevent minor problems from escalating into costly outages. This Enterprise RA includes alerting across Kubernetes, GPU, CPU and networking metrics, ensuring operations teams are quickly informed of critical issues impacting reliability and performance. With Prometheus collecting metrics, firing alerts, and Alertmanager routing these events as emails, infrastructure teams can stay on top of potential problems and safeguard infrastructure health.

Configuring Grafana for Alerting

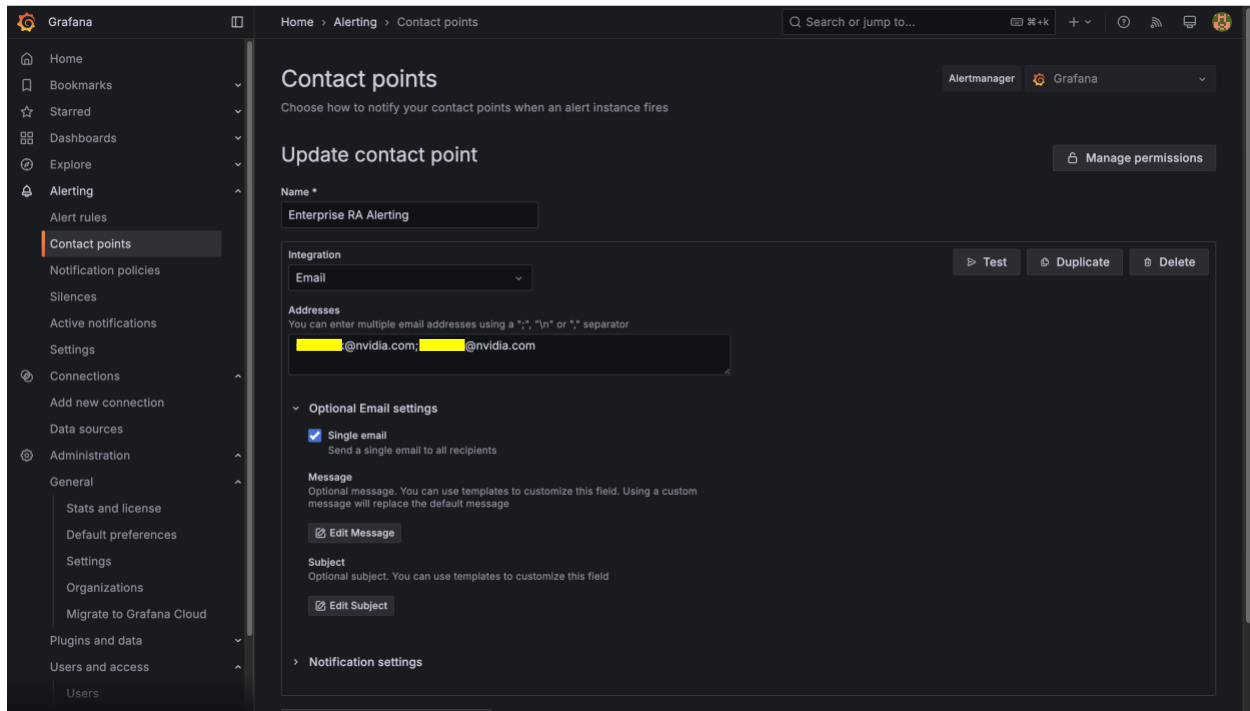
Folder Creation

Creating folders in Grafana is important for not only organizing alerts but giving alert groups a unique identifier. To create a folder in the Grafana using the UI, navigate to the **Dashboards** page from the main sidebar menu. Click on the **New** button on the top right and select **New Folder** from the dropdown menu. You should get a screen similar to the image below where you can enter a unique name for the folder – Grafana automatically generates a unique UID for every new folder. After creating it, you can find the folder's UID in its settings or in the URL. The UID will be leveraged in our configuration file below and is important for automating alert provisioning.



Contact Point Creation

Contact Points in Grafana make your monitoring actionable, allowing your team to respond quickly to incidents and escalate problems before they impact users. To create a contact point using the Grafana UI select **Contact Points** under Alerting on the left-hand menu. Click on **Create Contact Point** to open the setup dialog. As shown in the image below, give the contact point a name and input the addresses that are supposed to receive notifications. For the purposes of this Enterprise RA, we will only be notifying through e-mail so under Integration select the Email option. Under Optional Email settings, you can also edit the message or subject line for a specific contact point.



Setting up SMTP Server for configuration

To configure SMTP for Grafana when running on Kubernetes, first locate the ConfigMap managing the Grafana.ini file in your monitoring or Prometheus namespace through the command below.

Example 18. Command to find the Grafana ConfigMap

```
root@eratme-433-bcm-01:~# kubectl get cm -n prometheus
...
kube-prometheus-stack-grafana          1          80d
```

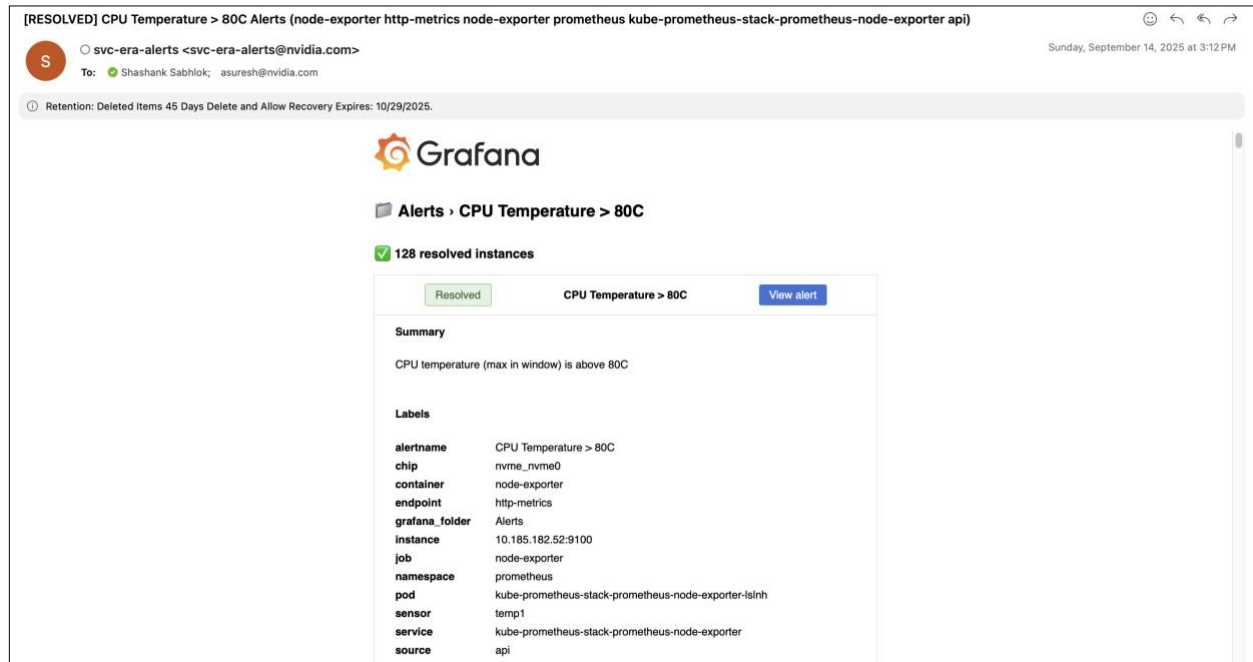
Once located, add your SMTP server settings (such as the host name, from_address and password) so your Grafana application can send e-mails to the addresses you have specified under your contact points. If the [smtp] section has not been created, make sure to add it as the last entry of the grafana.ini section as shown in the example below.

Example 19. Command to modify the Grafana ConfigMap with vim

```
root@eratme-433-bcm-01:~# kubectl edit cm kube-prometheus-stack-grafana -n prometheus

apiVersion: v1
data:
  grafana.ini: |
    [analytics]
    check_for_updates = true
    [grafana_net]
    url = https://grafana.net
    [log]
    mode = console
    [paths]
    data = /var/lib/grafana/
    logs = /var/log/grafana
    plugins = /var/lib/grafana/plugins
    provisioning = /etc/grafana/provisioning
    [server]
    domain = ''
    root_url = %(protocol)s://%(domain)s:%(http_port)s/grafana/
    serve_from_sub_path = true
    [smtp]
    enabled = true
    host = smtp.client.com:XX
    skip_verify = true
    from_address = era-alerts@client.com
    password = "hifromthe3rateam"
    from_name = Grafana Alerts
kind: ConfigMap
...
```

After saving your changes, make sure you restart the Grafana deployment for changes to take effect. A screenshot example of an alert from Grafana is below



Automated Alert Uploads

As alert rule count grows, manually setting up alerts in the Grafana UI or using repetitive curl commands can be time-consuming and lead to configuration inconsistencies. To simplify the process, this Enterprise RA leverages a Python script that reads alert configurations from JSON files and bulk uploads them into Grafana ([available on GitHub](#)). This workflow streamlines alert provisioning, helps ensure consistency, and makes it easier to manage changes across large-scale environments. The following sections outline how to use the script, along with its main features and benefits for operational teams.



Note: We use a Python upload script from [GitHub](#) to post alerts directly to the target folder in Grafana, rather than copying JSON files into a provisioned path such as `/etc/grafana/provisioning/alerting` inside the Grafana Kubernetes pod with persistent volume. This latter method has resulted in inconsistent behavior, especially in Grafana OSS setups.

Script Overview

This Enterprise RA script organizes its configuration using a centralized config.yaml file (as shown in the example below). All key details—including the Grafana server URL, API authentication token, default folder UID and alert rule JSON file paths need to be specified in this YAML file, allowing for easy updates and portability. Administrators can choose to list individual alert JSONs or enable a folder scan via the json_folder parameter, streamlining the selection process when large numbers of rules are involved.

Example 20. Parameters available in config.yaml

```
grafana_url: "https://YOUR IP ADDRESS:PORT"
api_token: "YOUR API TOKEN"
disable_provenance: true
default_folder_uid: "hifromthe3rateam"
json_files:
  - "gpu_alert_temp.json"
  - "gpu_alert_util.json"
  - "gpu_alert_pwr.json"
  - "cpu_temp.json"
  - "cpu_util.json"
  - "tensor_core_util.json"
  - "k8s_kubelet_down.json"
  - "k8s_apiserver_down.json"
  - "k8s_pending_pods.json"
  - "k8s_failed_pods.json"
  - "k8s_node_down.json"
  - "k8s_cpu_namespace.json"
  - "k8s_mem_namespace.json"
  - "storage_disk_util.json"
#json_folder: "./my_json_alerts"
```

Once the configuration has been modified, run the script using the Python command: `python grafana_upload.py`. Python must be installed on your system; most Linux environments include Python by default, and installation packages are available at python.org. Once configured, the script iterates through each alert rule JSON, uploading it to Grafana via API and providing immediate feedback on the outcome of each upload. The script checks for duplicate alert UIDs to prevent redundant entries and prints clear status messages indicating the success or failure of each operation. This structured approach ensures reliable bulk uploads, enables efficient alert management and reduces the risk of inconsistencies in production environments.

Example 21. Excerpt from running the `graf_upload.py` script to bulk upload alert JSONs

```
root@eratme-433-bcm-01:~# python graf_upload.py
Checking existing Grafana alert rules ...

/usr/local/lib/python3.10/dist-packages/urllib3/connectionpool.py:1097:
InsecureRequestWarning: Unverified HTTPS request is being made to host
'10.185.182.61'. Adding certificate verification is strongly advised. See:
https://urllib3.readthedocs.io/en/latest/advanced-usage.html#tls-warnings

    warnings.warn(

Uploading 13 alert JSON files to
https://10.185.182.61:30443/grafana/api/v1/provisioning/alert-rules ...

SKIPPED 'cpu_util.json': Alert rule 'CPU Utilization > 80%' already exists in folder
'dey37zy9mhurkb'.

SKIPPED 'k8s_apiserver_down.json': Alert rule 'Kubernetes - API Server Down' already
exists in folder 'dey37zy9mhurkb'.

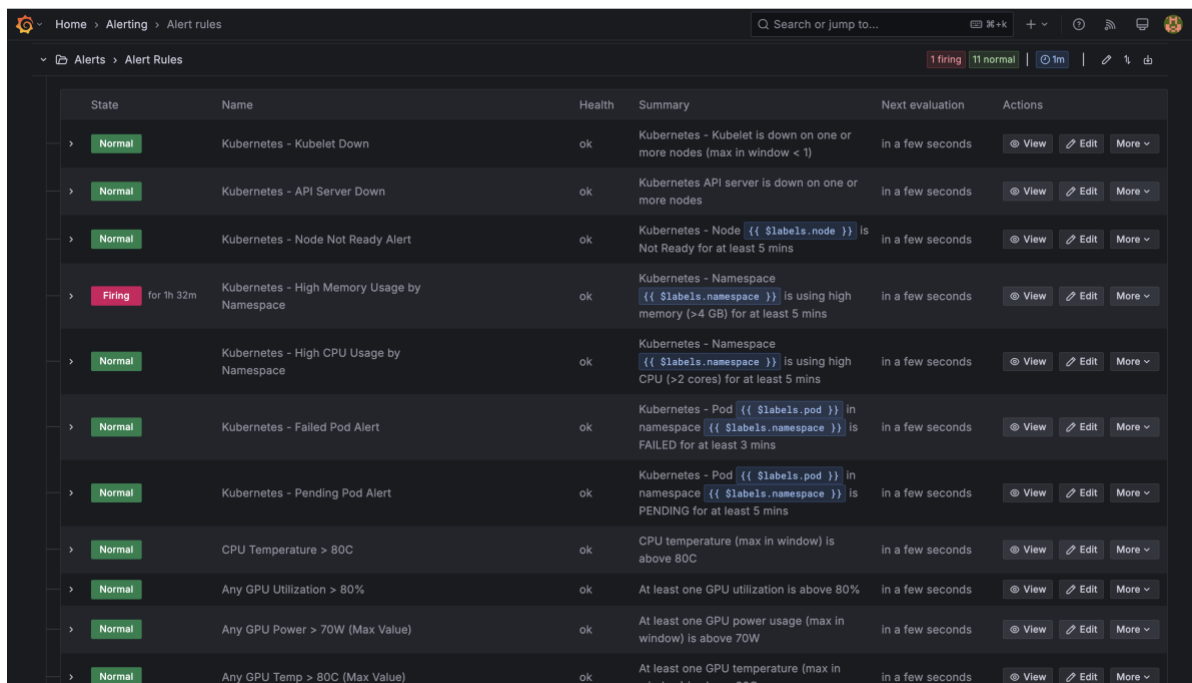
    warnings.warn(

Uploading: k8s_kubelet_down.json ... SUCCESS (rule id: 41, uid: feysyft9r89a8c)

    warnings.warn(

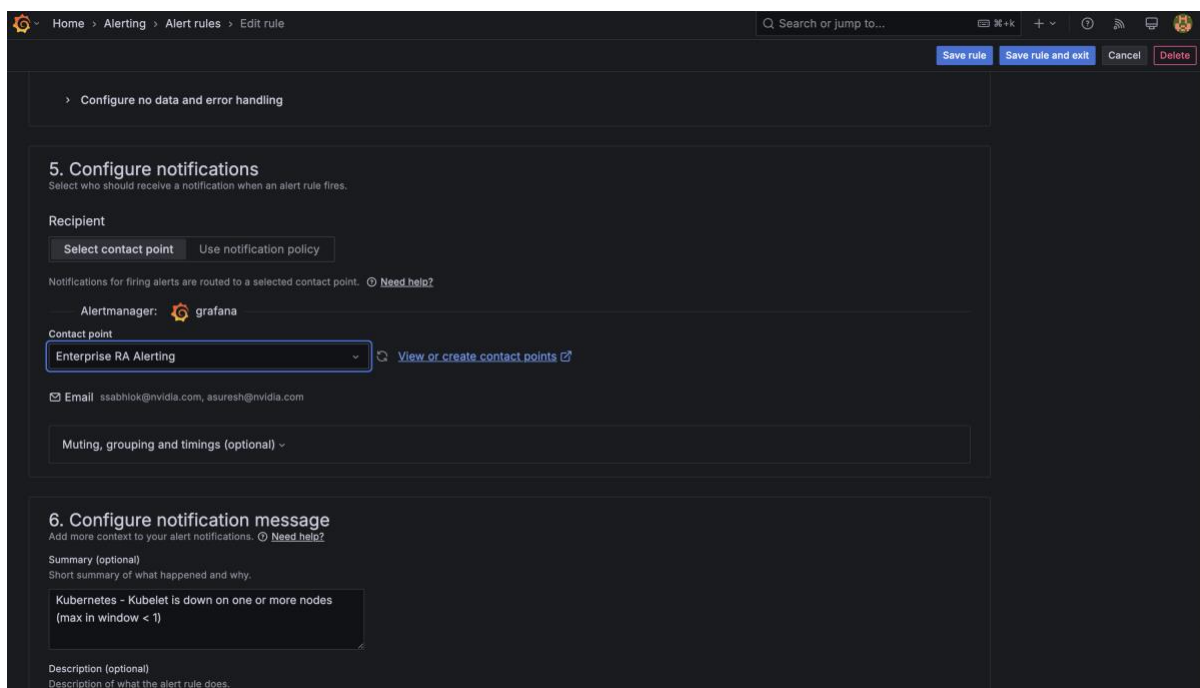
Uploading: tensor_core_util.json ... FAILED: invalid alert rule: no queries or
expressions are found
```

Once all the alerts have been bulk uploaded, you can expect your new created Alerts folder to look similar to the figure below:



State	Name	Health	Summary	Next evaluation	Actions
Normal	Kubernetes - Kubelet Down	ok	Kubernetes - Kubelet is down on one or more nodes (max in window < 1)	in a few seconds	View Edit More
Normal	Kubernetes - API Server Down	ok	Kubernetes API server is down on one or more nodes	in a few seconds	View Edit More
Normal	Kubernetes - Node Not Ready Alert	ok	Kubernetes - Node <code>{{ \$labels.node }}</code> is Not Ready for at least 5 mins	in a few seconds	View Edit More
Firing for 1h 32m	Kubernetes - High Memory Usage by Namespace	ok	Kubernetes - Namespace <code>{{ \$labels.namespace }}</code> is using high memory (>4 GB) for at least 5 mins	in a few seconds	View Edit More
Normal	Kubernetes - High CPU Usage by Namespace	ok	Kubernetes - Namespace <code>{{ \$labels.namespace }}</code> is using high CPU (>2 cores) for at least 5 mins	in a few seconds	View Edit More
Normal	Kubernetes - Failed Pod Alert	ok	Kubernetes - Pod <code>{{ \$labels.pod }}</code> in namespace <code>{{ \$labels.namespace }}</code> is FAILED for at least 3 mins	in a few seconds	View Edit More
Normal	Kubernetes - Pending Pod Alert	ok	Kubernetes - Pod <code>{{ \$labels.pod }}</code> in namespace <code>{{ \$labels.namespace }}</code> is PENDING for at least 5 mins	in a few seconds	View Edit More
Normal	CPU Temperature > 80C	ok	CPU temperature (max in window) is above 80C	in a few seconds	View Edit More
Normal	Any GPU Utilization > 80%	ok	At least one GPU utilization is above 80%	in a few seconds	View Edit More
Normal	Any GPU Power > 70W (Max Value)	ok	At least one GPU power usage (max in window) is above 70W	in a few seconds	View Edit More
Normal	Any GPU Temp > 80C (Max Value)	ok	At least one GPU temperature (max in window) is above 80C	in a few seconds	View Edit More

For each alert, specify the contact point to ensure notifications reach the right stakeholders in your stack, whether for hardware or Kubernetes components. This allows team members to receive relevant alerts based on their role. In the example below, the `k8s_kubelet_down` alert is configured through the Grafana UI to fire alerts at the Enterprise RA Alerting contact point.



Home > Alerting > Alert rules > Edit rule

Search or jump to...

Save rule Save rule and exit Cancel Delete

Configure no data and error handling

5. Configure notifications

Select who should receive a notification when an alert rule fires.

Recipient

Select contact point Use notification policy

Notifications for firing alerts are routed to a selected contact point. [Need help?](#)

Alertmanager: grafana

Contact point

Enterprise RA Alerting [View or create contact points](#)

Email ssabhlok@nvidia.com, asuresh@nvidia.com

Muting, grouping and timings (optional)

6. Configure notification message

Add more context to your alert notifications. [Need help?](#)

Summary (optional)

Short summary of what happened and why.

Kubernetes - Kubelet is down on one or more nodes (max in window < 1)

Description (optional)

Description of what the alert rule does.

Alert Rule JSONs

The script supports a wide range of alert scenarios, with each alert defined in a separate JSON file. Common alert types include monitoring metrics such as CPU usage, GPU utilization, memory availability, disk space, and network latency. Each JSON file contains all necessary parameters for Grafana to assess the corresponding metric and trigger alerts based on the health and status of your infrastructure.

To update or customize an alert rule, open the corresponding JSON file in a text editor and modify key parameters as required. Adjust the threshold fields to set when the alert should trigger—for example, setting `"threshold": 80` causes the alert to fire if average CPU utilization exceeds 80%. Edit the `"expr"` field to define the metric or logic being evaluated, tailoring it to your data source or monitoring strategy. The `"for"` field at the bottom of the JSON specifies how long the condition must be true before the alert fires (e.g., `"for": "1m"` means one minute). Additionally, each alert JSON should include a `"folderUID"` at the top to align with the structure of POST requests into Grafana OSS, although the alert ultimately appears in the folder indicated in `config.yaml` as described earlier.

Example 22. A JSON example for the GPU Temperature Alert

```
{
  "orgID": 1,
  "folderUID": "hifromthe3rateam",
  "ruleGroup": "Alert Rules",
  "title": "Any GPU Temp > 80C (Max Value)",
  "condition": "C",
  "data": [
    {
      "refId": "A",
      "queryType": "",
      "relativeTimeRange": {
        "from": 600,
        "to": 0
      },
      "datasourceUid": "prometheus",
      "model": {
        "expr": "DCGM_FI_DEV_GPU_TEMP",
        "interval": "",
        "intervalMs": 1000,
        "legendFormat": "",
        "editorMode": "code",
        "maxDataPoints": 43200,

```



```

    "refId": "A",
    "instant": true,
    "datasource": {
      "type": "prometheus",
      "uid": "prometheus"
    }
  },
  {
    "refId": "C",
    "queryType": "",
    "relativeTimeRange": {
      "from": 0,
      "to": 0
    },
    "datasourceUid": "__expr__",
    "model": {
      "type": "threshold",
      "expression": "A",
      "refId": "C",
      "conditions": [
        {
          "evaluator": {
            "type": "gt",
            "params": [80]
          },
          "operator": {
            "type": "and"
          },
          "query": {
            "params": ["A"]
          },
          "reducer": {
            "type": "max",
            "params": []
          },
          "type": "query"
        }
      ]
    }
  }
]

```

```

    }
  ],
  "datasource": {
    "type": "__expr__",
    "uid": "__expr__"
  },
  "intervalMs": 1000,
  "maxDataPoints": 43200
}
}
],
"noDataState": "NoData",
"execErrState": "Error",
"for": "1m",
"labels": {
  "source": "api"
},
"annotations": {
  "summary": "At least one GPU temperature (max in window) is above 80C"
},
"isPaused": false
}

```

Table 5. JSON alerts available on the Enterprise RA program [GitHub](#)

Stack Layer	File Name	Description (Alert if...)
Kubernetes	k8s_pending_pods.json	A pod is in pending state for at least 5 minutes
Kubernetes	k8s_failed_pods.json	Pods has failed for at least 3 minutes
Kubernetes	k8s_node_down.json	A Kubernetes node has been down for at least 5 minutes

Kubernetes	kube_apiserver_down.json	The Kubernetes API Server is down for at least 1 minute
Kubernetes	k8s_kubelet_down.json	The kubelet has been down for at least 5 minutes
Kubernetes	k8s_cpu_namespace.json	A namespace in Kubernetes is occupying more than 2 CPU cores for at least 5 minutes
Kubernetes	k8s_mem_namespace.json	A namespace in Kubernetes is occupying more than 4 GB memory for at least 5 minutes
GPU	gpu_alert_util.json	A GPU is utilized more than 80% for at least a minute
GPU	gpu_alert_pwr.json	The power consumption of any GPU exceeds 70W for at least a minute
GPU	gpu_alert_temp.json	Any GPU temperature exceeds 80 Celsius for at least a minute
CPU	cpu_util.json	On average, any CPU is utilized for more than 80% for at least a minute
CPU	cpu_temp.json	Any CPU temperature exceeds 80 Celsius for at least a minute

Disk Storage	<code>storage_disk_util.json</code>	Total bytes on a filesystem that are currently free/unallocated goes below 10 bytes
Disk Write Time	<code>disk_write_time.json</code>	Any disk has a read time greater than 10 seconds
Disk Read Time	<code>disk_read_time.json</code>	Any disk has a read time greater than 5 seconds
Port Down Status	<code>network_port_down.json</code>	Any switch interface is down for more than 1 minute
Unreachable Switch	<code>network_unreachable_switch.json</code>	Any switch is unreachable for more than 1 minute
BGP Peer State Change	<code>network_bgp_peer_state_change.json</code>	Any BGP peer that was recently Established moves to a non-Established state (Idle/Active) for more than 1 minute

For networking-related alerts such as Port Down Status, specify the UID of the NetQ VictoriaMetrics TSDB in the alert JSON (`datasourceUid` field), because it is different from the standard Prometheus data source UID used by other alerts



Note: Alert thresholds and time intervals in the table are example values; adjust them for your cluster setup and always reference the appropriate product datasheet or specifications when configuring. For example, for RTX Pro 6000 Servers BSE GPUs reference the product specification (NVOnline: 1117304) to get the Thermal Qualification Temperatures for the GPU Temperature Alert.

Summary

This Observability Guide for NVIDIA Enterprise RAs delivers a structured framework to monitor the full AI infrastructure stack, capturing telemetry across GPUs, CPUs, networking and workloads. Using NVIDIA tooling like DCGM, BCM, NIM Operator, and NetQ all integrated with Prometheus and Grafana, it enables real-time visibility and configurable alerting. This allows teams to proactively track performance, resource usage, and system health helping reduce troubleshooting time and optimize AI workloads at scale.

By adopting the Observability Guide for NVIDIA Enterprise RAs, organizations avoid the trouble of setting up their monitoring solutions from scratch. Operators gain deep visibility into the impact of AI inferencing workloads on their infrastructure, enabling more predictable scaling. With this comprehensive view, enterprises are equipped to maintain high service levels and confidently grow their AI infrastructure as their business evolves. The guide and tooling are compatible with multiple enterprise Kubernetes distributions, allowing customers to deploy the Kubernetes platform of their choice while retaining consistent monitoring and alerting capabilities.

Out of Scope

The following items are out of scope for this design document and will be addressed in subsequent releases:

- NetQ Host Metrics
- Logging

Appendix A

Table 6. Details of host ports that should be kept open for observability

Tool	Port Number
Kube Prometheus	9090
BCM Prometheus	8081
Grafana	80/3000
NIM	8000
OLTP ingress	30008
NetQ	32708